# Ingegneria del Software

Nicolò Giuliani

## Indice

**1 Modulo 1** **3**
1.1 Introduzione al corso . . . 3
1.2 La produzione del software . . . 3
1.2.1 Il software come prodotto . . . 3
1.2.2 Prodotti, sistemi e servizi . . . 3
1.2.3 Requisiti e feature . . . 4
1.2.4 Dipendenze . . . 4
1.2.5 Agenti e orchestratori . . . 4
1.2.6 Aspetti economici . . . 4
1.2.7 Garanzie e protezione legale . . . 5
1.2.8 Rischi e qualità . . . 5
1.2.9 Aspetti sociali . . . 6
1.3 Essence: gli Alpha del Kernel . . . 6
1.3.1 Le carte degli Alpha . . . 6
1.3.2 Relazioni tra gli Alpha . . . 15
1.4 Introduzione a Scrum . . . 16
1.4.1 Contesto: sviluppo agile . . . 16
1.4.2 Che cos'è Scrum . . . 16
1.4.3 Persone e ruoli . . . 17
1.4.4 Pilastri e valori . . . 17
1.4.5 Il processo: Product Backlog e Sprint . . . 18
1.4.6 Quadro d'insieme . . . 19
1.4.7 Scalabilità . . . 20
1.4.8 Considerazioni finali . . . 20
1.5 User stories . . . 20
1.5.1 Progettazione agile vs. tradizionale . . . 20
1.5.2 Il modello di user story . . . 21
1.5.3 Definition of Done . . . 21
1.5.4 Le US come frammenti di una conversazione . . . 21
1.5.5 Storie buone: INVEST . . . 21
1.5.6 La stima: story points e Planning Poker . . . 22
1.5.7 Backlog . . . 22
1.5.8 Strumenti per progetti agili . . . 23
1.5.9 Cucumber e Gherkin . . . 23

**2 Modulo 2** **24**
2.1 Controllo di versione e Git . . . 24
2.1.1 Il problema . . . 24
2.1.2 Version Control System . . . 24
2.1.3 Storia di Git . . . 24
2.1.4 Glossario minimo . . . 24
2.1.5 Il ciclo di lavoro base . . . 24
2.1.6 Tornare indietro . . . 25
2.1.7 Ignorare file: .gitignore . . . 25
2.1.8 Repository remoti . . . 26
2.1.9 Social coding . . . 26
2.1.10 Conflitti . . . 27
2.1.11 Git come filesystem . . . 27
2.1.12 I branch . . . 27
2.1.13 Merge e rebase . . . 28
2.1.14 Stash . . . 28
2.1.15 Tag . . . 28
2.1.16 Fork e merge/pull request . . . 28
2.1.17 Flussi di lavoro . . . 29
2.1.18 Tracciare la produttività . . . 29
2.2 Introduzione UML . . . 31
2.2.1 Introduzione al OO . . . 31
2.2.2 Definizione dei termini . . . 32
2.2.3 Dettagli UML . . . 32
2.2.4 Modellazione concetto OO . . . 33
2.2.5 Analisi OO . . . 34
2.3 OO Design in UML . . . 45
2.3.1 L'idea della progettazione a oggetti (OOD) . . . 45
2.3.2 Problematiche di progetto . . . 46
2.3.3 OOA vs. OOD . . . 46
2.3.4 Diagrammi di stato (Statechart) . . . 48
2.3.5 Diagrammi di attività . . . 50
2.3.6 Diagrammi di sequenza . . . 51
2.3.7 Diagrammi di collaborazione . . . 52
2.3.8 Esempio: mettere insieme tutti i pezzi . . . 52
2.3.9 Sistemi e sottosistemi . . . 54
2.3.10 Diagrammi dei package . . . 54
2.3.11 Design pattern . . . 55
2.3.12 Componenti e nodi . . . 56
2.3.13 UML per sistemi real-time . . . 56
2.3.14 Object Constraint Language (OCL) . . . 57
2.3.15 Refactoring . . . 58

# 1 Modulo 1

## 1.1 Introduzione al corso

Il corso presenta metodi e strumenti di **analisi**, **modellazione**, **progettazione** e **misura** utili allo sviluppo di prodotti software, con particolare riguardo ai metodi di sviluppo agili.

I nove argomenti principali dell'ingegneria del software sono: requisiti, design, costruzione, processo di sviluppo, testing, evoluzione, qualità, strumenti e metodi, gestione del progetto e gestione delle configurazioni.

Creare software fatto bene è difficile: è difficile progettarlo, scriverlo, leggerlo, descriverlo, modificarlo, misurarlo e decidere se è fatto bene.

## 1.2 La produzione del software

### 1.2.1 Il software come prodotto

Il software può essere definito in quattro modi complementari:

- **Opera dell'ingegno**: prodotto invisibile, intangibile, facilmente duplicabile ma costosissimo, protetto dalle leggi sul diritto d'autore.
- **Servizio digitale**: ha un'interfaccia (API), si basa su un'infrastruttura hw/sw/virtuale e garantisce per contratto una specifica qualità del servizio.
- **Componente di un sistema di elaborazione**: può essere di larga diffusione (*off the shelf*) oppure commissionato da un singolo committente.
- **Macchina astratta**: trasforma il computer realizzando funzioni utili per qualche scopo; ha un'architettura (struttura del codice) e un comportamento.

Il software è un prodotto **speciale** perché:

- è invisibile e intangibile;
- ogni prodotto ha molte dipendenze;
- è facilmente duplicabile e distribuibile in rete;
- in Europa non è brevettabile (ma è protetto dal diritto d'autore);
- il software di consumo non è garantito;
- viene acquisito su licenza (proprietaria, public domain, open source).

### 1.2.2 Prodotti, sistemi e servizi

- **Prodotti generici** (OTS, *off the shelf*): creati da un produttore e venduti sul mercato a molti clienti. Es. un videogioco.
- **Sistemi commissionati** (customizzati): commissionati da un cliente specifico e sviluppati apposta da un fornitore. Es. il portale dell'Università.
- **Servizi in perpetuo sviluppo**: offrono servizi 24/7 in continuo cambiamento. Es. Facebook, Amazon, chatbot.

Categorie (nuove) di software: apps ed ecosistemi software, servizi software, nuovi strumenti di sviluppo, social software, scraping/mining di big data, software embedded e IoT, software AI-native (agenti, chatbot, orchestratori).

Un **ecosistema software** è un mercato in cui si vendono prodotti (AppStore, PlayStore) o componenti e servizi (Amazon Elastic Computing): una collezione di prodotti software su una piattaforma definita da un'azienda, sviluppati ed evoluti nello stesso ambiente.

### 1.2.3 Requisiti e feature

- **Requisito software**: funzione o qualità controllabile (testabile) che deve possedere l'implementazione di un prodotto software. È importante per il *cliente*.
- **Feature software**: insieme di funzioni che permettono di usare un prodotto software in un servizio o prodotto. È importante per il *fornitore*.

Esempio: i *requisiti* di un servizio di commercio elettronico sono che l'utente possa registrarsi, aggiungere o togliere elementi al carrello, specificare indirizzi alternativi, pagare; la *feature* è il carrello per negozio elettronico (es. opencart).

### 1.2.4 Dipendenze

Ogni prodotto software dipende da altri prodotti software, che a loro volta dipendono da altro software. A ciascun prodotto si associa un **grafo delle dipendenze**, i cui nodi sono pacchetti software (es. librerie) in diverse versioni.

### 1.2.5 Agenti e orchestratori

- **Agente**: entità autonoma che prende decisioni ed esegue compiti specifici sfruttando l'intelligenza artificiale.
- **Orchestratore**: sistema che coordina, monitora e gestisce il flusso di lavoro di più agenti o strumenti.
- **Modello di collaborazione**: gli agenti operano in modo verticale sulla singola attività, mentre l'orchestratore garantisce la visione d'insieme e l'allineamento degli obiettivi.

### 1.2.6 Aspetti economici

Gli aspetti economici di un prodotto software sono: ambito d'uso (dimensione in righe), piano di sviluppo (durata), sforzo di sviluppo (costo), qualità (difetti) e produttività del team.

Il software è sempre il prodotto di un **processo di sviluppo**, che inizia con un'idea e termina quando il software viene ritirato. Il costo di sviluppo tende a crescere in proporzione al *quadrato* delle dimensioni: gli assistenti AI abbassano il costo di scrivere ogni singola riga, ma non quello di capire, integrare e mantenere un sistema che cresce, perché la curva quadratica riguarda la complessità delle interazioni tra le parti.

Due leggi classiche:

- **Legge di Wirth**: «il software diventa lento più rapidamente di quanto l'hardware diventi più veloce».
- **Legge di Eagleson**: qualsiasi codice che non hai rivisto negli ultimi sei mesi o più ti sembrerà estraneo, come se l'avesse scritto qualcun altro.

### 1.2.7 Garanzie e protezione legale

- La **verifica** garantisce l'aderenza a una specifica.
- La **validazione** garantisce l'accettazione da parte del cliente.
- La **certificazione** garantisce l'aderenza a specifiche definite dalla legge.

Il software commerciale di solito viene venduto senza garanzie (*as is*): nel software di consumo in teoria NON c'è alcuna garanzia, come dichiarato dal contratto di licenza mostrato al primo uso.

Il software è un'opera dell'ingegno: chi lo produce ha diritto a un compenso e copiarlo abusivamente è illegale; in Italia la legge 248/2000 punisce con il carcere da 6 mesi a 3 anni chi duplica abusivamente software.

Il **software libero** (non gratis) garantisce quattro libertà:

0. il programma può essere eseguito per qualsiasi scopo;
1. il programma può essere studiato e modificato per adattarlo a nuove esigenze;
2. il programma può essere liberamente distribuito;
3. il programma può essere liberamente migliorato e i miglioramenti possono essere liberamente distribuiti.

### 1.2.8 Rischi e qualità

I rischi riguardano lo sviluppo dei sistemi software, i difetti nel software operativo, l'esercizio dei sistemi software e le allucinazioni durante la generazione AI.

Esempi di incidenti: l'aggiornamento difettoso di CrowdStrike Falcon Sensor (19 luglio 2024) ha bloccato circa 8,5 milioni di sistemi Windows, con danni globali stimati in almeno 10 miliardi di dollari; un difetto nel sistema di gestione dei dati di volo di NATS ha bloccato quasi 3.000 voli e circa 150.000 passeggeri.

Alcune qualità del software:

- **Robustezza**: capacità di sopportare errori durante l'esecuzione.
- **Sostenibilità**: software duraturo, modificabile economicamente.
- **Riproducibilità**: controllo delle versioni.

Difetti rilevati durante l'esercizio:

- peggiori sistemi militari: 55 difetti/KLoC;
- migliori sistemi militari: 5 difetti/KLoC;
- sviluppo agile (XP): 1,4 difetti/KLoC;
- Apache web server (open source): 0,5 difetti/KLoC;
- NASA Space Shuttle: 0,1 difetti/KLoC.

### 1.2.9 Aspetti sociali

Lo sviluppo software è un lavoro di collaborazione: gli sviluppatori operano in team interdisciplinari, dove comunicazione efficace, empatia e comprensione delle diverse prospettive sono cruciali. Il software è costruito per le persone, quindi la *user experience* e il feedback continuo sono essenziali. Il software non è solo codice: è una costruzione sociale che influenza e viene influenzata dalle persone che lo sviluppano e lo usano. Chi lo sviluppa ha una responsabilità etica, perché il software non è neutrale.

## 1.3 Essence: gli Alpha del Kernel

**Essence** (standard OMG) definisce un *kernel* di concetti comuni a qualunque progetto software. Gli elementi essenziali da tenere sotto controllo si chiamano **Alpha** (*Abstract-Level Progress Health Attribute*): ogni Alpha attraversa una sequenza di **stati**, e ciascuno stato ha una lista di *checkpoint* da verificare. Gli Alpha sono rappresentati da carte (*Alpha State Cards*) usate dal team per valutare lo stato di avanzamento.

I 7 Alpha sono raggruppati in tre aree di interesse (*areas of concern*):

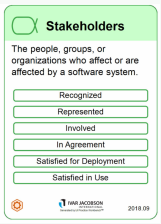
- **Customer**: Stakeholders, Opportunity;
- **Solution**: Requirements, Software System;
- **Endeavor**: Team, Work, Way of Working.

### 1.3.1 Le carte degli Alpha

Le carte ufficiali (*Alpha State Cards*, Ivar Jacobson International, 2018), con accanto la traduzione dei *checkpoint*. Il colore indica l'area di interesse: verde = Customer, giallo = Solution, blu = Endeavor.

**Stakeholders (Customer)**

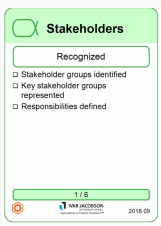
**Stakeholders**




Le persone, i gruppi o le organizzazioni che influenzano o sono influenzati da un sistema software.

1. Recognized — riconosciuti
2. Represented — rappresentati
3. Involved — coinvolti
4. In Agreement — d'accordo
5. Satisfied for Deployment — soddisfatti per il rilascio
6. Satisfied in Use — soddisfatti nell'uso

**Recognized — 1/6**



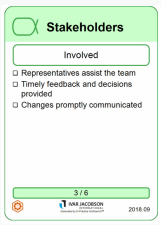

- ☐ Gruppi di stakeholder identificati
- ☐ Gruppi di stakeholder chiave rappresentati
- ☐ Responsabilità definite

### Represented — 2/6




- ☐ Responsabilità concordate
- ☐ Rappresentanti autorizzati
- ☐ Approccio collaborativo concordato
- ☐ Way of working supportato e rispettato

### Involved — 3/6



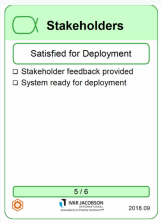

- ☐ I rappresentanti assistono il team
- ☐ Feedback e decisioni forniti per tempo
- ☐ Cambiamenti comunicati prontamente

### In Agreement — 4/6



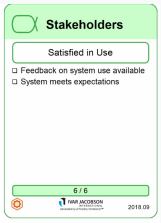

- ☐ Aspettative minime concordate
- ☐ I rappresentanti sono soddisfatti del proprio coinvolgimento
- ☐ Il contributo dei rappresentanti è valorizzato
- ☐ Il contributo del team è valorizzato
- ☐ Priorità chiare e prospettive bilanciate

### Satisfied for Deployment — 5/6




- ☐ Feedback degli stakeholder fornito
- ☐ Sistema pronto per il rilascio

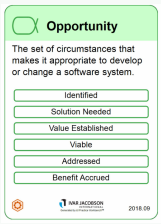
### Satisfied in Use — 6/6




- ☐ Feedback sull'uso del sistema disponibile
- ☐ Il sistema soddisfa le aspettative

## Opportunity (Customer)

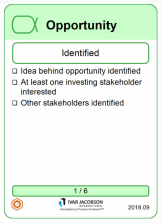
## Opportunity




L'insieme di circostanze che rende appropriato sviluppare o modificare un sistema software.

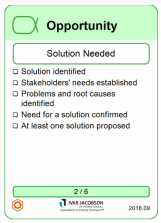
1. Identified — identificata
2. Solution Needed — serve una soluzione
3. Value Established — valore stabilito
4. Viable — perseguibile
5. Addressed — affrontata
6. Benefit Accrued — benefici maturati

## Identified — 1/6




- ☐ Idea dietro l'opportunità identificata
- ☐ Almeno uno stakeholder investitore interessato
- ☐ Altri stakeholder identificati

## Solution Needed — 2/6



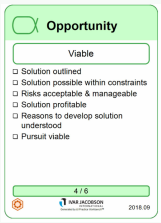

- ☐ Soluzione identificata
- ☐ Bisogni degli stakeholder stabiliti
- ☐ Problemi e cause radice identificati
- ☐ Necessità di una soluzione confermata
- ☐ Almeno una soluzione proposta

## Value Established — 3/6




- ☐ Valore dell'opportunità quantificato
- ☐ Impatto della soluzione compreso
- ☐ Valore del sistema compreso
- ☐ Criteri di successo chiari
- ☐ Risultati attesi chiari e quantificati

## Viable — 4/6




- ☐ Soluzione delineata
- ☐ Soluzione possibile entro i vincoli
- ☐ Rischi accettabili e gestibili
- ☐ Soluzione profittevole
- ☐ Ragioni per sviluppare la soluzione comprese
- ☐ Il perseguimento è sostenibile

## Addressed — 5/6




- ☐ Opportunità affrontata
- ☐ Soluzione degna di essere rilasciata
- ☐ Stakeholder soddisfatti

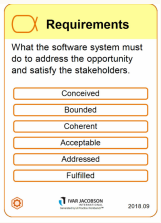
### Benefit Accrued — 6/6




- ☐ La soluzione produce benefici
- ☐ ROI accettabile

## Requirements (Solution)

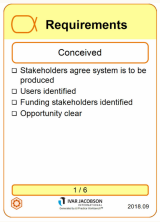
### Requirements




Ciò che il sistema software deve fare per sfruttare l'opportunità e soddisfare gli stakeholder.

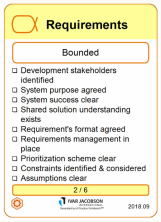
1. Conceived — concepiti
2. Bounded — delimitati
3. Coherent — coerenti
4. Acceptable — accettabili
5. Addressed — affrontati
6. Fulfilled — soddisfatti

### Conceived — 1/6



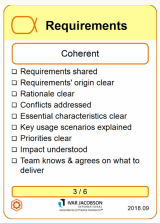

- ☐ Gli stakeholder concordano che il sistema va prodotto
- ☐ Utenti identificati
- ☐ Stakeholder finanziatori identificati
- ☐ Opportunità chiara

### Bounded — 2/6



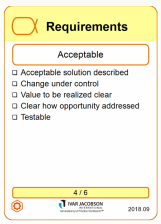

- ☐ Stakeholder dello sviluppo identificati
- ☐ Scopo del sistema concordato
- ☐ Successo del sistema chiaro
- ☐ Esiste una comprensione condivisa della soluzione
- ☐ Formato dei requisiti concordato
- ☐ Gestione dei requisiti attiva
- ☐ Schema di prioritizzazione chiaro
- ☐ Vincoli identificati e considerati
- ☐ Assunzioni chiare

### Coherent — 3/6



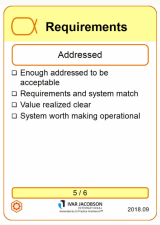

- ☐ Requisiti condivisi
- ☐ Origine dei requisiti chiara
- ☐ Motivazione chiara
- ☐ Conflitti risolti
- ☐ Caratteristiche essenziali chiare
- ☐ Scenari d'uso chiave spiegati
- ☐ Priorità chiare
- ☐ Impatto compreso
- ☐ Il team sa e concorda su cosa consegnare

### Acceptable — 4/6



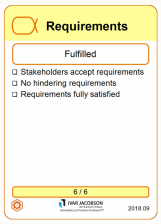

- ☐ Soluzione accettabile descritta
- ☐ Cambiamento sotto controllo
- ☐ Valore da realizzare chiaro
- ☐ Chiaro come viene sfruttata l'opportunità
- ☐ Testabile

### Addressed — 5/6




- ☐ Affrontati abbastanza da essere accettabili
- ☐ Requisiti e sistema coincidono
- ☐ Valore realizzato chiaro
- ☐ Sistema degno di essere reso operativo

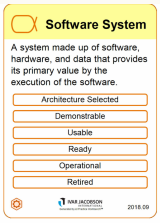
### Fulfilled — 6/6




- ☐ Gli stakeholder accettano i requisiti
- ☐ Nessun requisito di ostacolo
- ☐ Requisiti pienamente soddisfatti

## Software System (Solution)

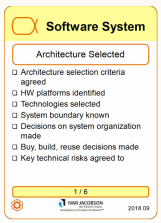
### Software System




Un sistema fatto di software, hardware e dati, che fornisce il suo valore primario attraverso l'esecuzione del software.

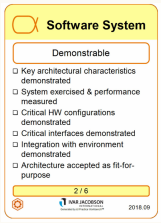
1. Architecture Selected — architettura scelta
2. Demonstrable — dimostrabile
3. Usable — usabile
4. Ready — pronto
5. Operational — operativo
6. Retired — dismesso

### Architecture Selected — 1/6



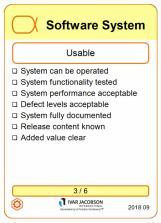

- ☐ Criteri di scelta dell'architettura concordati
- ☐ Piattaforme hardware identificate
- ☐ Tecnologie selezionate
- ☐ Confine del sistema noto
- ☐ Decisioni sull'organizzazione del sistema prese
- ☐ Decisioni *buy, build, reuse* prese
- ☐ Rischi tecnici chiave concordati

## Demonstrable — 2/6



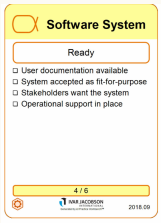

- ☐ Caratteristiche architetturali chiave dimostrate
- ☐ Sistema esercitato e prestazioni misurate
- ☐ Configurazioni hardware critiche dimostrate
- ☐ Interfacce critiche dimostrate
- ☐ Integrazione con l'ambiente dimostrata
- ☐ Architettura accettata come adeguata allo scopo

## Usable — 3/6



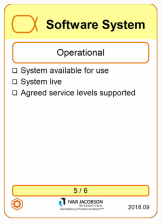

- ☐ Il sistema può essere messo in esercizio
- ☐ Funzionalità del sistema testate
- ☐ Prestazioni del sistema accettabili
- ☐ Livelli di difetti accettabili
- ☐ Sistema completamente documentato
- ☐ Contenuto della release noto
- ☐ Valore aggiunto chiaro

## Ready — 4/6



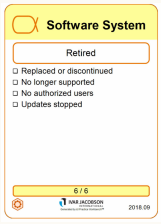

- ☐ Documentazione utente disponibile
- ☐ Sistema accettato come adeguato allo scopo
- ☐ Gli stakeholder vogliono il sistema
- ☐ Supporto operativo predisposto

## Operational — 5/6




- ☐ Sistema disponibile all'uso
- ☐ Sistema in esercizio (*live*)
- ☐ Livelli di servizio concordati supportati

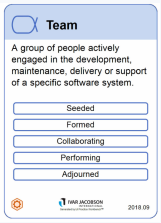
## Retired — 6/6




- ☐ Sostituito o dismesso
- ☐ Non più supportato
- ☐ Nessun utente autorizzato
- ☐ Aggiornamenti interrotti

### Team (Endeavor)

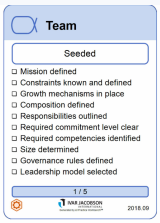
## Team




Un gruppo di persone attivamente impegnate nello sviluppo, manutenzione, consegna o supporto di uno specifico sistema software.

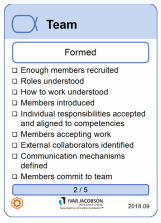
1. Seeded — avviato
2. Formed — formato
3. Collaborating — collaborante
4. Performing — performante
5. Adjourned — sciolto

## Seeded — 1/5



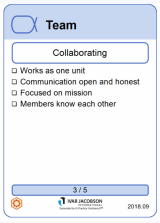

- ☐ Missione definita
- ☐ Vincoli noti e definiti
- ☐ Meccanismi di crescita predisposti
- ☐ Composizione definita
- ☐ Responsabilità delineate
- ☐ Livello di impegno richiesto chiaro
- ☐ Competenze richieste identificate
- ☐ Dimensione determinata
- ☐ Regole di governance definite
- ☐ Modello di leadership scelto

## Formed — 2/5



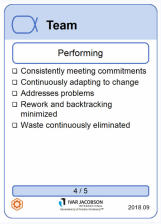

- ☐ Membri reclutati a sufficienza
- ☐ Ruoli compresi
- ☐ Modo di lavorare compreso
- ☐ Membri presentati
- ☐ Responsabilità individuali accettate e allineate alle competenze
- ☐ I membri accettano il lavoro
- ☐ Collaboratori esterni identificati
- ☐ Meccanismi di comunicazione definiti
- ☐ I membri si impegnano verso il team

## Collaborating — 3/5



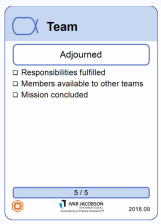

- ☐ Lavora come un'unica unità
- ☐ Comunicazione aperta e onesta
- ☐ Focalizzato sulla missione
- ☐ I membri si conoscono tra loro

## Performing — 4/5




- ☐ Rispetta costantemente gli impegni
- ☐ Si adatta di continuo al cambiamento
- ☐ Affronta i problemi
- ☐ Rilavorazioni e passi indietro minimizzati
- ☐ Sprechi eliminati di continuo

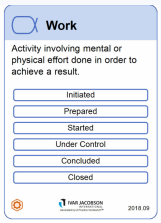
### Adjourned — 5/5




- ☐ Responsabilità assolte
- ☐ Membri disponibili per altri team
- ☐ Missione conclusa

## Work (Endeavor)

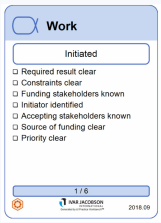
### Work




Attività che comporta sforzo mentale o fisico svolto per ottenere un risultato.

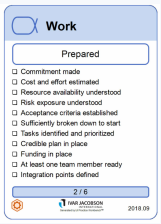
1. Initiated — avviato
2. Prepared — preparato
3. Started — iniziato
4. Under Control — sotto controllo
5. Concluded — concluso
6. Closed — chiuso

### Initiated — 1/6




- ☐ Risultato richiesto chiaro
- ☐ Vincoli chiari
- ☐ Stakeholder finanziatori noti
- ☐ Iniziatore identificato
- ☐ Stakeholder che accettano il risultato noti
- ☐ Fonte del finanziamento chiara
- ☐ Priorità chiara

### Prepared — 2/6



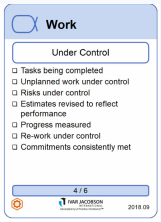

- ☐ Impegno assunto
- ☐ Costo e sforzo stimati
- ☐ Disponibilità di risorse compresa
- ☐ Esposizione al rischio compresa
- ☐ Criteri di accettazione stabiliti
- ☐ Scomposto abbastanza da poter iniziare
- ☐ Task identificati e prioritizzati
- ☐ Piano credibile predisposto
- ☐ Finanziamento disponibile
- ☐ Almeno un membro del team pronto
- ☐ Punti di integrazione definiti

### Started — 3/6



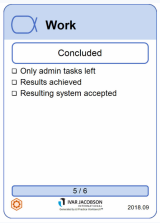

- ☐ Sviluppo iniziato
- ☐ Avanzamento monitorato
- ☐ *Definition of done* stabilita
- ☐ Task in avanzamento

### Under Control — 4/6



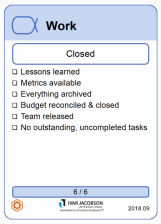

- ☐ Task in via di completamento
- ☐ Lavoro non pianificato sotto controllo
- ☐ Rischi sotto controllo
- ☐ Stime riviste in base alle prestazioni
- ☐ Avanzamento misurato
- ☐ Rilavorazioni sotto controllo
- ☐ Impegni rispettati con costanza

### Concluded — 5/6




- ☐ Restano solo task amministrativi
- ☐ Risultati raggiunti
- ☐ Sistema risultante accettato

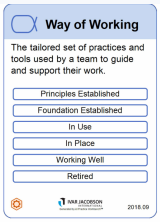
### Closed — 6/6




- ☐ Lezioni apprese
- ☐ Metriche disponibili
- ☐ Tutto archiviato
- ☐ Budget riconciliato e chiuso
- ☐ Team rilasciato
- ☐ Nessun task in sospeso o incompleto

## Way of Working (Endeavor)

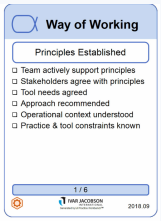
### Way of Working




L'insieme di pratiche e strumenti, adattato al contesto, usato dal team per guidare e supportare il proprio lavoro.

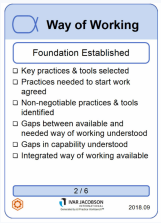
1. Principles Established — principi stabiliti
2. Foundation Established — fondamenta stabilite
3. In Use — in uso
4. In Place — consolidato
5. Working Well — funziona bene
6. Retired — dismesso

### Principles Established — 1/6



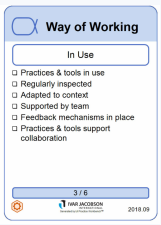

- ☐ Il team sostiene attivamente i principi
- ☐ Gli stakeholder concordano con i principi
- ☐ Esigenze di strumenti concordate
- ☐ Approccio raccomandato
- ☐ Contesto operativo compreso
- ☐ Vincoli di pratiche e strumenti noti

**Foundation Established — 2/6**



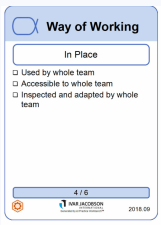

- ☐ Pratiche e strumenti chiave selezionati
- ☐ Pratiche necessarie per iniziare concordate
- ☐ Pratiche e strumenti non negoziabili identificati
- ☐ Divari tra way of working disponibile e necessario compresi
- ☐ Divari di capacità compresi
- ☐ Way of working integrato disponibile

**In Use — 3/6**



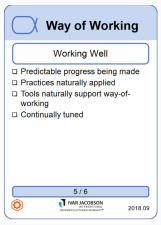

- ☐ Pratiche e strumenti in uso
- ☐ Ispezionati regolarmente
- ☐ Adattati al contesto
- ☐ Supportati dal team
- ☐ Meccanismi di feedback predisposti
- ☐ Pratiche e strumenti supportano la collaborazione

**In Place — 4/6**




- ☐ Usato da tutto il team
- ☐ Accessibile a tutto il team
- ☐ Ispezionato e adattato da tutto il team

**Working Well — 5/6**




- ☐ Avanzamento prevedibile
- ☐ Pratiche applicate con naturalezza
- ☐ Gli strumenti supportano naturalmente il way of working
- ☐ Messo a punto di continuo

**Retired — 6/6**




- ☐ Non più in uso
- ☐ Lezioni apprese condivise

### 1.3.2 Relazioni tra gli Alpha

Gli Alpha non sono indipendenti: gli *Stakeholders* forniscono l'*Opportunity* e usano il *Software System*, che a sua volta soddisfa (*fulfils*) i *Requirements*; l'*Opportunity* si focalizza sui *Requirements*, che delimitano (*scope and constrain*) il *Work*; il *Team* produce il *Software System* ed esegue il *Work*, guidato dal *Way of Working*.

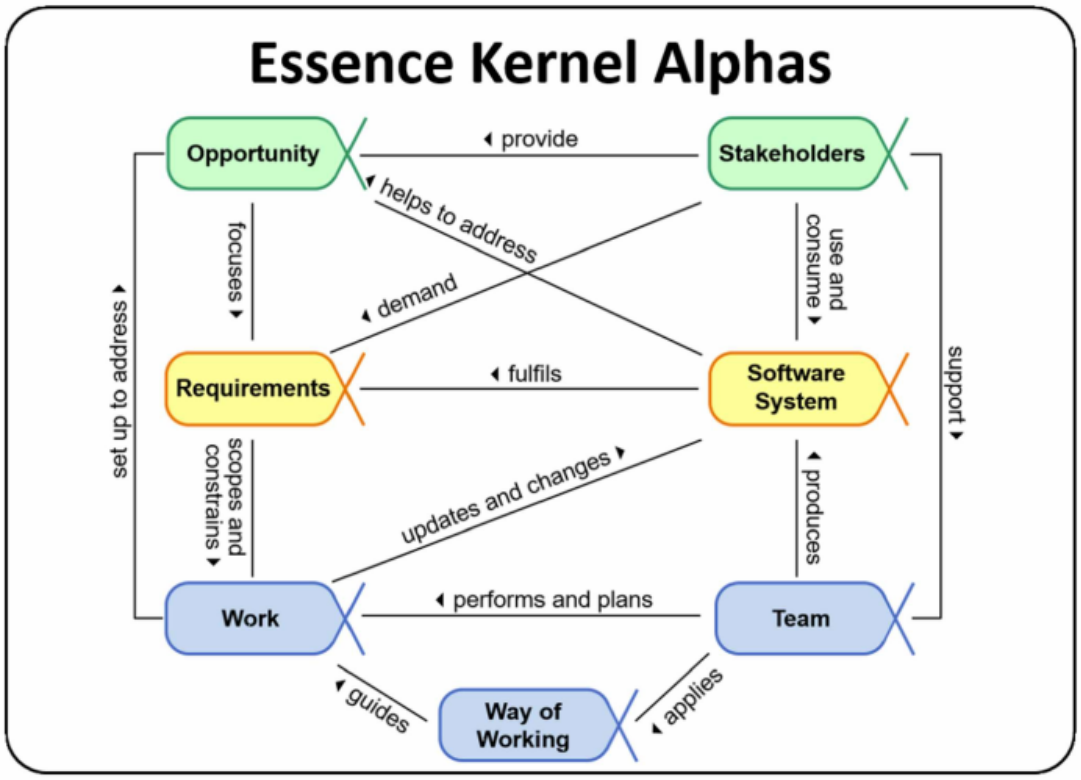



Figura 1: Il grafo degli Alpha del kernel di Essence.

## 1.4 Introduzione a Scrum

### 1.4.1 Contesto: sviluppo agile

Certi sviluppi non si pianificano in anticipo: il problema è complesso e richiede il contributo di molte persone, la soluzione all'inizio non è chiara, i requisiti cambieranno durante lo sviluppo, il lavoro può essere articolato in *incrementi* e serve una collaborazione stretta con feedback rapido e continuo degli utenti finali.

Il **Manifesto agile** dichiara di preferire:

- individui e interazioni $\succ$ processi e strumenti;
- software che funziona $\succ$ documentazione completa;
- collaborazione del cliente $\succ$ negoziazione contrattuale;
- rispondere al cambiamento $\succ$ seguire un piano.

**Dalla staffetta al pacchetto di mischia.** Takeuchi e Nonaka (1986) contrappongono l'approccio a *staffetta* (requisiti $\rightarrow$ design $\rightarrow$ codice $\rightarrow$ test, una fase alla volta) a quello *rugby*, in cui il team avanza come un'unità passandosi la palla. Il team Scrum fa un po' di tutto (requisiti, design, codice, test) in ogni sprint.

### 1.4.2 Che cos'è Scrum

Scrum è un **framework di project management** per lo sviluppo *iterativo e incrementale* di prodotti, nato per i progetti software. Non è un processo né una tecnica: è un contenitore in cui usare processi e tecniche diverse (spesso insieme a XP). Rende evidente l'efficacia delle proprie

pratiche di product management e sviluppo, così da migliorarle: non porta all'eccellenza, ma *mostra il livello di inefficienza* (Ken Schwaber).

In sintesi:

- il cliente può ispezionare versioni funzionanti del software ogni 1–4 settimane;
- il cliente definisce le funzioni e le priorità; il team decide ogni giorno come realizzare al meglio quelle a priorità più alta;
- a ogni sprint nasce una nuova versione, esaminata per decidere se fare un altro sprint o produrre un rilascio;
- team **auto-organizzante**, metodo agile parzialmente pianificato;
- i requisiti stanno nel **Backlog**; sviluppo guidato da storie e test;
- il prodotto cresce in *sprint* di durata fissa;
- meeting: review e retrospettive.

### 1.4.3 Persone e ruoli

Un team ha $5 \pm 2$ persone (in altre slide $7 \pm 2$), tutte egualmente responsabili della riuscita del progetto. I ruoli sono tre. Si distingue tra ruoli *pig* (coinvolti e responsabili del risultato: team, ScrumMaster, Product Owner) e ruoli *chicken* (stakeholder esterni, coinvolti ma non responsabili) dalla storiella della gallina e del maiale.

| **Product Owner (PO)** | **Scrum Master (SM)** | **Development Team (DT)** |
|---|---|---|
| Rappresenta il cliente | Difensore del team | Cross-funzionale |
| Definisce le feature | Elimina gli ostacoli | Colocalizzato |
| Stabilisce le priorità | Evita le interruzioni | Auto-organizzato |
| Fissa le date di rilascio | Facilitatore del team | A tempo pieno |
| Fornisce feedback | Controllore del processo | Definisce i task, stima lo sforzo |
| Gestisce gli stakeholder | Gestisce il management | Evolve il processo |
| Accetta o rifiuta i risultati | | Assicura la qualità |

### 1.4.4 Pilastri e valori

**I tre pilastri** guidano tutte le attività:

- **Trasparenza**: gli aspetti significativi del processo devono essere visibili ai responsabili del risultato e compresi allo stesso modo (linguaggio comune; la definizione di "Fatto" condivisa da chi esegue e da chi accetta).
- **Ispezione**: gli artefatti vanno ispezionati spesso per rilevare deviazioni dall'obiettivo dello sprint, ma non così spesso da intralciare il lavoro, e da chi ne ha la competenza.
- **Adattamento**: se l'ispezione rileva aspetti fuori dai limiti accettabili, processo o materiale vanno adattati il più rapidamente possibile.

**I cinque valori.**

- **Focus**: SM rimuove impedimenti; PO massimizza il valore gestendo il Product Backlog; il Team sviluppa in funzione della Definition of Done.
- **Coraggio**: proteggere e guidare il Team (SM), fidarsi e dialogare con gli stakeholder (PO), superare i propri limiti in ottica *inspect-and-adapt* (Team).
- **Apertura**: aperti ai cambiamenti interni ed esterni e a nuove soluzioni, soprattutto tecniche.
- **Impegno** (commitment): il DT lo dichiara a ogni Sprint Planning; l'SM diffonde la cultura Scrum; il PO si impegna con gli stakeholder a un incremento costante.
- **Rispetto**, nell'ottica della *Prima Direttiva delle Retrospettive* (Kerth): qualunque cosa si scopra, si crede che ognuno abbia fatto del suo meglio date le conoscenze, le capacità, le risorse e la situazione del momento.

### 1.4.5 Il processo: Product Backlog e Sprint

Il **Product Backlog** è la lista prioritizzata delle user story (anche come *user story map*, backlog bidimensionale). Il processo si compone di **Sprint** di durata costante (2–4 settimane, per migliorare il ritmo); ogni sprint include design, codifica e test, estrae dal product backlog storie *Ready* e aggiunge al prodotto codice *Done*. Il risultato è un **incremento di prodotto potenzialmente rilasciabile** (MVP): verificato, testato, rifattorizzato, allineato alla DoD.

**Sprint planning.** Pianificazione di gruppo (PO, SM, Team) di almeno un'ora per ogni settimana di sprint.

- il Team sceglie, insieme al PO, le user story del product backlog che diventano lo **sprint backlog** e fissa lo **sprint goal** (breve descrizione del lavoro dello sprint);
- PO e Team concordano la *Definition of Done* per gli elementi dello sprint;
- il Team stima quanto riuscirà a fare, secondo la propria *velocità*;
- si studia l'architettura di alto livello, si definiscono i task e se ne stima lo sforzo in ore (tecnica: **Planning Poker**).

**Durante lo sprint non si cambia**: è compito dello SM pianificare le durate in modo che il team non cambi. Il PO può solo *cancellare* lo sprint se l'obiettivo diventa obsoleto (mercato o organizzazione cambiati) o palesemente irraggiungibile: evento raro e negativo, di solito si evita.

**Ready e Done (non coincidono).**

- Una storia è **Ready** se: tutto il Team la capisce; è dettagliata abbastanza da essere testata; il PO ne comprende il valore e può prioritizzarla; il Team può stimarla e completarla in uno sprint; è indipendente.
- **Done** è concordata da Team e PO all'inizio, ad esempio: codifica completata, test di unità scritti ed eseguiti, test di integrazione e prestazionali superati, documentazione minimale scritta. La storia non è finita finché non soddisfa il PO.

**Scrumboard (Taskboard / Kanban).** Lo sprint backlog sta su un tabellone modificabile da ogni membro.

- il lavoro non è assegnato ma *richiesto volontariamente* dai membri;
- completata una storia, la si sposta nella colonna corrispondente;
- la stima dello sforzo residuo si aggiorna ogni giorno;
- il lavoro "emerge" sul tabellone: se un lavoro è poco chiaro si crea un item con stima maggiore e lo si scompone dopo (scelta del DT, non del PO).

**Daily Scrum (stand-up).** Max 15 minuti, in piedi, niente problem solving; chiunque può assistere, ma parlano solo i *pig*. Tre domande: cosa ho fatto ieri, cosa farò oggi/domani, che problemi mi aspetto. Le risposte sono promesse ai compagni, non report per lo SM.

**Burndown chart.** Monitora l'andamento dello sprint confrontando il lavoro rimanente ideale con quello effettivo (ore residue per giorno), idealmente in modo automatico. Ogni giorno i membri ristimano il tempo rimasto sul proprio compito (es. 6 h spese su un task da 10 h e altre 6 stimate: 2 h oltre la stima). Esempio: sprint di 2 settimane, 7 persone, 6 h/giorno $\Rightarrow$ 420 ore ideali. Il *cumulative flow diagram* è uno strumento affine.

**Sprint review.** Riflessione sul *prodotto*: cosa è stato o no completato, con una *demo* (niente slide) delle nuove funzioni; 2–4 ore; si verifica lo stato del product backlog; invitare tutti.

**Sprint retrospective.** Riflessione sul *processo*: cosa è andato bene, quali impedimenti; max 3 ore; partecipano SM, PO, Team e forse committenti e utenti. Obiettivi: esaminare persone, relazioni, processi e strumenti; rivedere i driver architetturali; individuare e ordinare cosa è andato bene e i miglioramenti possibili; creare un *improvement plan*. Formato più semplice: **Start / Stop / Continue**.

**Rilasci e backlog refinement.** Ogni organizzazione sceglie la cadenza di rilascio: a fine sprint, a fine di un gruppo di sprint, o a ogni feature (continuous delivery/deployment); conviene comunque il *release planning* a lungo termine. Nel **backlog refinement** PO e DT modificano, ri-stimano e riordinano il backlog secondo l'esperienza; non dovrebbe superare il 10% del tempo del team. Il PO può cambiare il backlog in ogni momento, *ma non lo sprint in corso*.

### 1.4.6 Quadro d'insieme

| **Rituali** | **Artefatti** | **Pratiche** |
|---|---|---|
| Sprint planning | Product backlog | Planning Poker |
| Stand-up meeting | Sprint goal | Testing |
| Sprint review | Sprint backlog | Pair programming |
| Sprint retrospective | Scrumboard | Continuous integration |
| Backlog refinement | Potentially shippable increment | |
| Release planning | Burndown chart, improvement plan | |

Scrum non impone pratiche di sviluppo; molti team adottano quelle di XP: **TDD** (spesso implica simple design, testing, refactoring, continuous integration; vale la piramide dei test) e **pair programming** (spesso implica collective ownership, settimana di 40 ore, coding standard).

### 1.4.7 Scalabilità

Con team di $7 \pm 2$ persone si scala su sistemi grandi tramite **scrum di scrum** (alcuni progetti hanno coinvolto oltre 500 persone). Fattori: tipo di applicazione, dimensione e dispersione dei team, durata del progetto. Configurazioni:

- *team unico, molti PO*: un solo SM e un solo team;
- *molti team, prodotto unico*: un Chief Product Owner sopra i PO, ogni team con il suo SM e PO.

Secondo Sutherland lo scrum di scrum *non* è un "meta-scrum": è un meccanismo operativo di consegna, responsabile di consegnare il software funzionante di tutti i team secondo la DoD a fine sprint; il suo Master ne risponde.

### 1.4.8 Considerazioni finali

**Problemi tipici:** ignoranza dei valori agili e di Scrum; software non testato a fine sprint (cattiva DoD); backlog non pronto all'inizio (cattiva Ready); mancata facilitazione; mancato supporto di manager e stakeholder; gestione caotica degli scrum di scrum.

**Patti tra stakeholder e team.** Il team promette: il PO difenderà gli interessi degli stakeholder; il loro tempo sarà usato bene; lavoro della massima qualità compatibilmente con i vincoli; nuove funzionalità validabili a ogni sprint. Gli stakeholder promettono: disponibilità ad aiutare; supporto allo SM nel rimuovere ostacoli; priorità e vincoli invariati durante lo sprint senza consenso del Team; partecipare a Scrum non danneggerà la carriera dei membri.

**XP o Scrum?**

| XP | Scrum |
|---|---|
| Orientato alla qualità (test driven) | Orientato al project management |
| Iterazione 1–2 settimane | Sprint 2–4 settimane |
| Requisiti sempre modificabili | Requisiti modificabili a fine sprint |
| Il cliente ordina le storie | Il team ordina le storie |
| Coaching informale | Scrum master certificato |
| TDD, pair programming, planning game, refactoring | Retrospettiva, strumenti di PM, planning poker |

XP e Scrum possono coesistere (e spesso lo fanno).

## 1.5 User stories

### 1.5.1 Progettazione agile vs. tradizionale

Nell'approccio tradizionale i requisiti stanno in un **SRS**: documento tecnico, legato ad aspetti legali, il più ampio e approfondito possibile, frutto di lunga preparazione e in pratica raramente modificato in corso d'opera. I progetti agili affrontano i requisiti in modo opposto (senza escludere gli approcci classici): si incentrano su due artefatti, le **user story** e il **product backlog**.

### 1.5.2 Il modello di user story

- *As a* ⟨tipo di utente⟩, *I want* ⟨fare qualcosa⟩, *so that* ⟨ottengo un valore⟩.
- Esempio: *come utente della biblioteca (ruolo) voglio consultare il catalogo (funzione) per prendere a prestito un volume (valore).*

Esiste un modello alternativo (più compatto) e i due **non sono equivalenti**: il primo va favorito come modello *iniziale*, più vicino al cliente e meno definito (la storia la deve capire lui); il secondo è preferibile per storie brevi, meglio ancora per sotto-storie, e può servire a definire i task tecnici che compongono la US. Nel processo agile si aggiungono i *criteri di accettazione* del valore di business.

Le storie hanno granularità variabile. La catena è: bisogni → user story → task/backlog → feature/tool → prodotto software.

**Condizioni e test.** Una storia si arricchisce di dettagli o test aggiuntivi, ad esempio per "consultare il catalogo": volume già in prestito, iscritto con troppi volumi, volume ordinato ma non arrivato, volume da acquistare; per l'iscrizione al portale: CV vuoto o molto lungo, stipendio mancante o a sei cifre.

**US e task.** Le US non sono e non devono essere tecniche (es. "implementare l'autenticazione con SSL"), ma possono essere esplicitate in **task** tecnici (es. autenticazione OAuth, upload del CV, web crawler, report, segnalazione all'utente). I nomi non sono standard: in certi ambienti *issue* o *ticket* indicano storie e/o task.

**Partizioni orizzontali e verticali.** Scrivere buone storie non è facile: è più semplice (e gli sviluppatori lo preferiscono, per efficienza) partizionare il backlog *orizzontalmente*, stratificando l'architettura. Le partizioni **verticali** tagliano invece una fetta di tutti gli strati e permettono di consegnare rapidamente singole funzionalità, con feedback continuo dal cliente.

### 1.5.3 Definition of Done

Serie di criteri *oggettivi* per stabilire se una storia è completata. È stabilita dal team di sviluppo, di solito uguale per tutto il progetto (può evolvere). Esempio: la storia è committata; esiste un unit test con coverage $\geq 75\%$; è commentata con Javadoc/Pydoc/JSdoc; segue le guidelines di coding del documento X.

### 1.5.4 Le US come frammenti di una conversazione

Ogni ruolo vi cerca una risposta: utente (come descrivo i miei desideri?), stakeholder (come faccio avere successo al prodotto?), PM (come traccio e pianifico?), BA (quali sono i dettagli?), UX (quali sono i bisogni?), developer (quali task oggi?), QA (come valido il task completato?).

### 1.5.5 Storie buone: INVEST

Come per i libri, ci sono storie buone e cattive. Per obiettivi sensati (**S.M.A.R.T.**: specifici, misurabili, accettati, realistici, temporizzati) si deve investire (**I.N.V.E.S.T.**) nelle user story:

- **Independent**: ogni storia può essere lavorata con pieno controllo, senza dipendere da altre.
- **Negotiable**: non sono scolpite nella pietra né perfette; il dettaglio può essere eccessivo e va verificato col cliente (alcune limitazioni però sono fisse).

- **Valuable**: devono mostrare il vantaggio per il cliente e gli altri partecipanti; servono a evitare le "storie tecniche".
- **Estimable**: dettaglio sufficiente per stimare il tempo; ogni team avrà una stima diversa (ma consistente). Scale: lineari, Fibonacci, T-shirt, ore; spesso Planning Poker.
- **Sized right**: completabili in un'iterazione. Più si è vicini al lavoro, più devono essere specifiche; si parte da storie grandi da spezzare (modello a piramide: *tema* → *epica* → *storia*).
- **Testable**: i *criteri di accettazione* fanno diventare realtà le storie e assicurano che il risultato sia quello previsto; i test andrebbero automatizzati e non si dovrebbe iniziare a codificare prima di averli chiari.

### 1.5.6 La stima: story points e Planning Poker

Tutte le US vanno stimate in **story points**: unità astratta e *relativa*, utile per confrontare le storie tra loro. La stima coinvolge tutti e incoraggia una discussione costruttiva.

Il **Planning Poker** (attribuito a Grenning, 2002) unisce esperienza, analogia e punti di vista diversi in modo rapido; partecipa tutto il team di sviluppo. Ogni partecipante ha un set di carte con scala pseudo-Fibonacci (esistono app e siti per il remoto). Procedura:

1. il moderatore espone la storia;
2. ognuno sceglie una carta senza mostrarla;
3. si scoprono tutte insieme;
4. gli *outlier* spiegano la loro scelta;
5. si ripete fino al consenso.

Trucco: come prima storia se ne sceglie una di difficoltà "media", altrimenti la scala potrebbe non bastare.

### 1.5.7 Backlog

L'insieme delle storie stimate e pronte costituisce il **backlog**, l'analogo agile dell'SRS. Le storie sono prioritizzate per dimensione, rischi, necessità; una selezione costituisce l'obiettivo di un'iterazione (sprint). Il backlog si aggiorna di continuo.

**Vantaggi e svantaggi delle US.**

| Pro | Contro |
|---|---|
| Cicli di feedback continui | Non adatte ad aspetti puramente tecnici (spike, bug) |
| Coinvolgimento del cliente, visione comune, conversazione | Problemi legali |
| Prioritizzazione | Si perde la visione globale (design povero) |
| Massimizzazione dell'usabilità | Difficili da scrivere |
| Facilita la stima dei tempi | |

### 1.5.8 Strumenti per progetti agili

- **Post-it, whiteboard, penna**: il metodo consigliato, favorisce l'interazione; problemi con team distribuiti e difficile da integrare nel software di gestione.
- **Trello**: bacheca elettronica general purpose, freemium (la parte gratuita basta), con estensioni per user story e sviluppo.
- **WeKan**: clone open source di Trello, con meno funzioni, tipicamente installato su server locale.
- **Pivotal Tracker**: specifico per user story e progetti agili; freemium, gratuito fino a 3 utenti.
- **Taiga**: gestione progetti per il mondo agile, open source (web o locale).
- **Big guns**: Atlassian Jira, Microsoft Teams, VersionOne: soluzioni complete, con curva di apprendimento ripida, proprietarie e costose.

### 1.5.9 Cucumber e Gherkin

**Cucumber** è uno strumento per il *Behavior Driven Development* (BDD), tecnica basata su TDD, OOP e domain-driven design: usa conversazione ed esempi concreti per costruire una comprensione condivisa del comportamento del programma. Si appoggia a **Gherkin**, linguaggio descrittivo vicino al linguaggio naturale, per scrivere script di test di alto livello che verificano il comportamento del sistema (usa regular expression e sistemi di build come Ant, Maven, Gradle, Make).

Struttura di un file `.feature`:

```
Feature: Is it Friday yet?
  In order to ...   As a ...   I want ...

  Scenario: Sunday isn't Friday
    Given today is Sunday
    When I ask whether it's Friday yet
    Then I should be told "Nope"
```

*Given* = precondizioni, *When* = azioni dell'attore, *Then* = risultato verificabile; *And* aggiunge passi. Esiste anche la versione italiana (`#language: it`: Funzionalità, Scenario, Dato, Quando, Allora, E). Per ogni passo si scrive in Java uno *step definition* con `@Given`, `@When`, `@Then`; finché non è implementato il passo risulta *undefined/pending*.

**Use case vs. user story.** Le user story sono più leggere e conversazionali; gli use case più formali e dettagliati (vedi il confronto nelle slide).

# 2 Modulo 2

## 2.1 Controllo di versione e Git

### 2.1.1 Il problema

Quando un progetto software diventa complesso, anche la sua gestione è complicata. Senza uno strumento di controllo delle revisioni:

- *programming in the small*: è difficile recuperare le versioni più vecchie basandosi sulla data ed è difficile stabilire le differenze tra le versioni di uno stesso file;
- *programming in the large/many*: non c'è modo di recuperare le versioni precedenti né di vederne le differenze, la fusione delle versioni è manuale, lenta e di dubbio successo, e sono possibili sovrascritture accidentali.

### 2.1.2 Version Control System

Un **VCS** (detto anche *versioning*, *revision control system* o *source code management*, SCM) è la gestione delle modifiche a documenti, programmi e altre informazioni memorizzate come file.

In pratica un VCS fornisce:

- un posto dove memorizzare le varie versioni del codice e dei documenti correlati (backup multiplo e flessibile);
- la possibilità di «tornare indietro» in caso di errore e di tracciare le modifiche;
- lo sviluppo parallelo facilitato, potendo lavorare su più versioni contemporaneamente.

Evoluzione storica: SCCS e RCS (1982) → CVS (1986) → Subversion (2000) → Git (2005).

### 2.1.3 Storia di Git

Git nasce dal kernel Linux. Fino al 2002 le modifiche al kernel venivano passate come file compressi; poi si iniziò a usare il DVCS proprietario **BitKeeper**. Nel 2005 ne fu revocato l'uso gratuito, e ciò indusse Linus Torvalds a sviluppare uno strumento proprio: dopo 15 giorni di sviluppo Git era pronto. Git è veloce ed efficiente anche per grandi progetti e ha un ottimo sistema di ramificazioni per lo sviluppo non lineare; oggi si stima lo usi almeno l'85% dei professionisti.

### 2.1.4 Glossario minimo

- **Repository** (deposito): un insieme di oggetti (file) e i riferimenti necessari, in pratica una «foto» del progetto. Può essere online o offline.
- **Clone**: l'atto di duplicare localmente un repository.
- **Commit**: l'invio al repository delle modifiche effettuate.
- **Checkout**: il ripristino di una particolare versione del progetto.

### 2.1.5 Il ciclo di lavoro base

Il flusso tipico attraversa gli stati dei file:

$$\text{Non tracciato} \xrightarrow{\texttt{git add}} \text{Staging area} \xrightarrow{\texttt{git commit}} \text{Tracciato / Non modificato}$$

**Configurazione iniziale**

- `git --version` — verifica dell'installazione.
- identificazione (one-shot), controllabile con `git config --list`:

```
git config --global user.name "Alex York"
git config --global user.email alex@example.com
```

- `git config --global init.defaultBranch main` — semplifica l'interazione con GitHub e GitLab.

**Comandi base**

- `git init` — inizializza il repository nella cartella corrente.
- `git add .` — aggiunge i file alla staging area.
- `git commit -m "messaggio"` — registra le modifiche (`-m` evita di finire nell'editor).
- `git commit -am "messaggio"` — commit veloce: add + commit per i file già tracciati.
- `git status` — stato dell'albero di lavoro.
- `git log` — storia delle modifiche: identificativo esadecimale, autore, data, commento, posizione di HEAD e del branch. Utile `git log --oneline`.
- `git reflog` — storia sintetica degli spostamenti di HEAD.

### 2.1.6 Tornare indietro

- `git checkout <checksum>` — ripristina *tutti* i file a una versione precedente e porta in stato di **detached HEAD**. `git checkout main` riporta HEAD all'ultimo commit. *Non committare in stato di detached HEAD*: si rischia di perdere il lavoro. `git switch -` torna al branch precedente.
- `git restore -s <checksum> <nomefile>` — ripristina un *solo* file dal passato (nelle vecchie versioni: `git checkout <checksum> -- nomefile`).
- `git revert <commit id>` — ricostruisce un commit precedente creando un *nuovo* commit: più sicuro perché non distrugge la storia, ma appesantisce il log.
- `git reset` — è l'*undo* di `git add` (toglie dalla staging area).
- `git reset --hard` — cancella le modifiche non committate e ripristina l'ultima versione salvata.

### 2.1.7 Ignorare file: .gitignore

Git si basa sulle differenze di testo con le versioni precedenti, quindi non funziona bene con i file binari (`.pdf`, `.png`, `.docx`, `.exe`, `.class`): ogni versione viene salvata integralmente, con perdita di spazio e di tempo.

Il file `.gitignore` (col punto iniziale) elenca file e percorsi che NON sono mai inseriti nella staging area. Esempio:

```
# ignora i file eseguibili e png
*.exe
*.png

# ignora tutti i file nelle cartelle indicate
temp/
img/
```

Lo switch `-f` (`--force`) elimina il controllo di `.gitignore`:
`git add temp/antipatico.txt -f`.

### 2.1.8 Repository remoti

Schema di base: `git clone` per duplicare un repository remoto, `git push` per inviare i commit, `git pull` per riceverli. Schema alternativo, partendo da un repo locale già esistente: `git remote add origin <URL>`, poi push e pull.

Sequenza tipica di collegamento a un remoto vuoto:

```
git config pull.rebase false
git remote add origin [URL]
git pull origin main --allow-unrelated-histories
git push -u origin --all --force
```

**Sicurezza** I protocolli remoti di Git usano spesso crittografia a chiave pubblica. Le chiavi si generano con:

```
ssh-keygen -t rsa -b 4096 -C "commento"
```

Il contenuto di `.ssh/id_rsa.pub` va incollato nelle impostazioni SSH del servizio remoto. La verifica si fa con `ssh -T git@gitlab.com`.

**Collaboratori** Se il repo è pubblico tutti possono clonarlo ma nessuno può modificarlo; se è privato nessuno lo può vedere. Per consentire la modifica occorre autorizzare esplicitamente gli utenti (*project members*).

### 2.1.9 Social coding

L'arrivo del Web 2.0 e il bisogno di condividere codice hanno portato ai primi siti di condivisione (Sourceforge, 1999), poi evoluti in vere comunità di social networking grazie alle potenzialità di networking di Git.

- **GitHub** (2008, comprato da Microsoft nel 2018): riferimento per community e open source, il più vasto ecosistema di progetti e sviluppatori, comprensivo di piattaforma DevOps.
- **GitLab** (2014): open source, ottimizzato per il modello DevOps, focus su DevSecOps dal repository alla CI/CD, sicurezza e deployment; disponibile anche *self-managed*, favorito dalle aziende con problemi di privacy.
- **Bitbucket** (Atlassian): orientato ai team che usano l'ecosistema Atlassian (Jira, Confluence, Bamboo); disponibile in cloud e come Data Center.

### 2.1.10 Conflitti

Un conflitto nasce quando due sviluppatori modificano la stessa porzione di file. Il push viene rifiutato (*rejected, fetch first*) e il successivo `git pull` genera il conflitto, segnalato nel file con i marcatori:

```
<<<<<<< HEAD
Un file di Bill Gates
=======
Un file di Alex York
>>>>>>> d19a0f36141bb4a03281330a0cf02cce7533c869
```

Metodi di risoluzione:

1. **Manuale**: editare il file rimuovendo i marcatori, poi `git add`, `git commit`, `git push`.
2. **Forzante**: `git reset --hard HEAD` e poi `git pull -X theirs` (accetta le modifiche remote) oppure `-X ours` (accetta le proprie).
3. **Distruttivo**: `git reset --hard HEAD` seguito da `git push -f`. Elimina per sempre le modifiche non accettate.

### 2.1.11 Git come filesystem

Linus Torvalds aveva in mente un filesystem quando lavorava su Git:

- `git rm` — equivale a `rm` di Unix;
- `git mv` — equivale a `mv`, serve anche a rinominare mantenendo il tracciamento;
- `git clean --force` — elimina tutti i file NON tracciati dalla directory di lavoro.

Le directory non esistono in Git: spariscono se sono vuote. Per tenerle la convenzione è inserire un file fittizio chiamato `.gitkeep`.

### 2.1.12 I branch

Un **branch** è una diramazione del codice: i commit relativi alla diramazione restano isolati rispetto alla versione principale (`main` o `master`), dando totale libertà di lavorare, provare e «sporcare» il codice senza compromettere il lavoro degli altri. Quando si è pronti si ricongiunge la diramazione al tronco principale, e solo in quel momento ci si occupa degli eventuali conflitti. I branch sono veloci da creare e occupano pochissimo spazio.

- `git checkout -b branch1` — crea il branch e vi si posiziona; da quel momento i comandi hanno effetto solo su quel branch.
- `git branch` — elenca i branch, marcando con `*` quello attivo.
- `git branch -d <nome>` — cancella un branch locale.
- `git push origin --delete <nome>` — cancella un branch remoto.
- `git branch -M <nome>` — rinomina un branch locale.
- `git push -u origin branch1` — fa tracciare il branch anche sul remoto.
- `git log --oneline --decorate --graph --all` — log grafico di tutti i branch.

### 2.1.13 Merge e rebase

Al termine di una porzione significativa di lavoro si ricongiungono i rami: l'operazione è il **merge** (fusione). Come per il pull (che è di fatto un merge) si possono generare conflitti da risolvere.

- **Merge** (`git checkout main; git merge branch2`): riceve nel ramo corrente i commit dell'altro ramo. Non è invasivo sulla struttura pre-esistente dei rami, ma il risultato può essere di non facile interpretazione per il team, perché servono numerosi merge per allineare il branch.
- **Rebase** (`git checkout feature; git rebase master`): accoda l'intero ramo `feature` a `master`, di fatto «riscrivendo la storia» dei commit. Il risultato è un flusso molto più lineare, ma si perde lo storico delle modifiche precedenti all'operazione: *mai eseguire un rebase su rami pubblici.*

Altre operazioni:

- `git cherry-pick` — «pesca» un commit da qualsiasi branch e lo applica al branch attuale.
- `git rebase -i HEAD~5` — sessione interattiva per comprimere gli ultimi cinque commit.
- `git merge --squash feature/login` — combina i commit del branch in un unico commit (da effettuare successivamente).
- `git commit --amend` — modifica l'ultimo commit, ideale per correzioni.

### 2.1.14 Stash

Se un merge sovrascriverebbe modifiche locali non committate, Git lo interrompe. Le soluzioni sono: committare le modifiche, eliminarle con `git reset --hard`, oppure «nasconderle sotto il tappeto» con `git stash`. Dopo il merge si recuperano con `git stash pop` (che può a sua volta generare conflitti) e si eliminano con `git stash drop`.

### 2.1.15 Tag

I tag «marcano» un particolare commit: milestone, fine sprint, eventi particolari.

- **Lightweight**: `git tag pippo`, semplice puntatore a un commit, analogo allo snapshot di un filesystem.
- **Annotated**: `git tag -a pluto -m "messaggio"`, ha un messaggio associato, può essere firmato; `git show pluto` mostra molte informazioni.

I tag *non* vengono pushati automaticamente: servono `git push origin <tagname>` oppure `git push origin --tags`.

### 2.1.16 Fork e merge/pull request

Il **forking** è una caratteristica dei siti di social coding che permette di duplicare un repository esistente inserendolo tra i propri progetti (con libertà totale di accesso) pur mantenendo un legame con il repository originario, detto **upstream**.

Su questa relazione si basa il sistema delle **pull request** (GitHub, Bitbucket) o **merge request** (GitLab): dopo aver forkato un repository e realizzato le modifiche, si invia una richiesta di inserimento invece di applicarla direttamente; il responsabile può accettarla o rifiutarla, eventualmente con un commento.

### 2.1.17 Flussi di lavoro

**Centralized Workflow** Il server remoto è il riferimento («custode della verità»). Ogni sviluppatore clona il repository e sviluppa localmente; terminata una sezione significativa pulla, integra localmente e reinvia al repo remoto.
*Vantaggi*: facile da capire, ottimo per team piccoli. *Svantaggi*: alla lunga gestire i conflitti diventa problematico, non sfrutta la natura distribuita di Git.

**Gitflow** Proposto da Vincent Driessen nel 2010. Prevede due branch di durata infinita:

- `master`/`main` — contiene il codice di produzione, pronto;
- `develop` — branch di sviluppo.

E branch temporanei con terminologia precisa:

- `feature-*` — nuove funzionalità, figlia di `develop`;
- `hotfix-*` — riparazione di errori del master, da mergiare in `master` o `develop`;
- `release-*` — preparazione di un rilascio, figlia di `develop`, da riunire a `master` e `develop`.

*Vantaggi*: ideale se occorrono rilasci regolari, presenza di tool, master con solo codice stabile. *Svantaggi*: la storia passata diventa intricata e illeggibile, non funziona bene con versioni multiple, rischio di «merge hell».

**Feature branch workflow** Il repo remoto è il riferimento. Ogni sviluppatore forka il repository e lo clona localmente; se è fuori sincronia si ribasa alla situazione corrente. Crea un branch locale (replicato sul server remoto) e, quando ha finito, chiede conferma ai colleghi prima del push.
*Vantaggi*: promuove la collaborazione e la proprietà condivisa, master resta stabile. *Svantaggi*: i branch più vecchi tendono ad avere problemi di merge.

**Forking workflow** Esiste un server *upstream* di riferimento. Ogni sviluppatore lo forka, clona localmente il proprio repository e lavora indisturbato; quando si sente pronto invia una pull request al *project master*.
*Vantaggi*: il repo upstream resta impeccabile grazie a un ferreo controllo degli accessi in scrittura. *Svantaggi*: più complicato, implica una struttura gerarchica.

### 2.1.18 Tracciare la produttività

- `git shortlog -sn --all` — numero di commit per autore.
- **git-quick-stats** — offre molte opzioni oltre al semplice conteggio.
- **Gitinspector** — genera un grafico riassuntivo.
- **Gource** — genera un'animazione dei commit (attenzione alle dimensioni del video).

Il numero di commit non è mai una metrica troppo significativa: «*Git mente*». La fotografia può essere infedele se gli autori non committano in prima persona ma delegano a un'unica persona, se si inseriscono librerie corpose o codice trovato in rete, o se si lavora in *pair/mob programming*. Rimedi:

1. **Manuale**: ogni autore committa solo il codice che ha davvero scritto, eventualmente modificando temporaneamente il proprio username con `git config`.

2. **Co-author**: nel messaggio di commit, dopo almeno una riga vuota, aggiungere la riga `Co-authored-by: nome <email>`. È rilevata da GitHub e GitLab, ma non da `git log` e Gitinspector.

3. **git-pair**: configurare gli utenti con `git users` e committare con `gitp commit -m "..."`.

## 2.2 Introduzione UML

Introduzione al Unified Modeling Language (UML). A cosa servono? Formalizzare l'idea del OO.

### 2.2.1 Introduzione al OO

E' un metodo di vedere il mondo delle applicazione, descrivere il modello delle applicazioni, un metodologia compresiva che permette di sviluppare sistemi di software, usare siili concetti in tutto la fase di sviluppo.
L'obbiettivo primario è l'alta qualità:

- Nascondere informazioni
- Astrazione
- Modularizzazione
- Riutilizzo

⇒ Un approccio OO forza lo sviluppatore ad applicare questi concetti.

**Idea chiave** Rappresenta il mondo in termini di **oggetti che interagiscono**; li rappresentiamo nel ciclo vita:

- Modelizzazione del concetto OO
- Analisi OO
- Design OO
- Programmazione OO

**Concetti chiave**

- Classi e gerarchie
  - attributi
  - metodi
  - ereditarietà
  - relazioni con altre classi
- Oggetti
  - attributi con valori defautl
  - relazioni stanziate
- Messaggi e metodi per rispondere ai messaggi

### 2.2.2 Definizione dei termini

**Classi** Una classe una ocllezione di oggetti simili definiti come:

- template
- descrizione generalizzate
- pattern
- schematiche ("blueprint")

Una classe definisce **proprietà (attributi)** che appartengono all'oggetto della classe e i suoi **comportamenti (operazioni)** di tutti gli oggetti della classe. Quando una classe viene definita una istanza specifica della classe può essere definita.

Una `operazione` è un eseguibile incapsulato nella classe che è designato a operare una o più classi.
L'`ereditarietà` è l'abilitò di definire una classe come estensione su un'altra. Una ereditarieà NON E' UNA istantazione.
Il `polimorfismo` è l'abilità di utilizzare lo stesso nome per metodi che fanno operazioni "simili" su differenti oggetti.

- Polimorfismo ad-hoc (overloading): diverse funzioni con lo stesso nome ma parametri differenti
- Polimorfismo generico: template generico che definisce una struttura comune a un set di classi
- Polimorfismo ereditario (overriding)

### 2.2.3 Dettagli UML

I diagrammi UML si dividono in due famiglie.

1. Class Diagram
2. Object Diagram
3. Component Diagram
4. Deployment Diagram

} **Structural Diagrams**

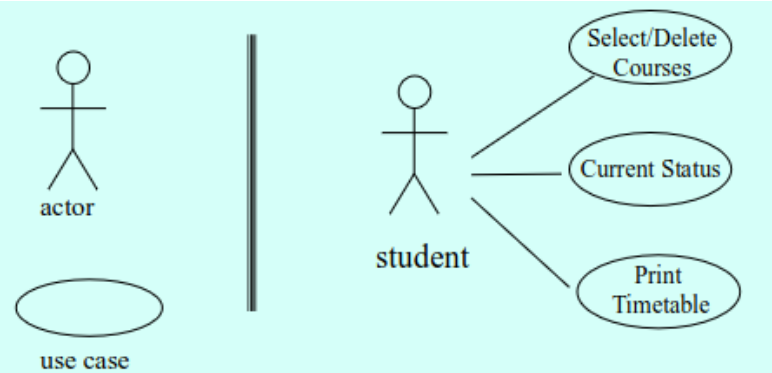
5. Use Case Diagram
6. Sequence Diagram
7. Collaboration Diagram
8. Statechart Diagram
9. Activity Diagram

} **Behavioral Diagrams**

### 2.2.4 Modellazione concetto OO

Obbiettivo: capire il contesto operazionale del sistma (OO context analysis), capire i requisiti reali del sistema (OO requirementi analysis).

**Use case** Un `Use case description` e' uno scenario che descrive un utilizzo di un sistema. Include un **diagramma** con attori che rappresentao ruoli o dispositivi; una descrizione testuale della sequenza delle attivita'.

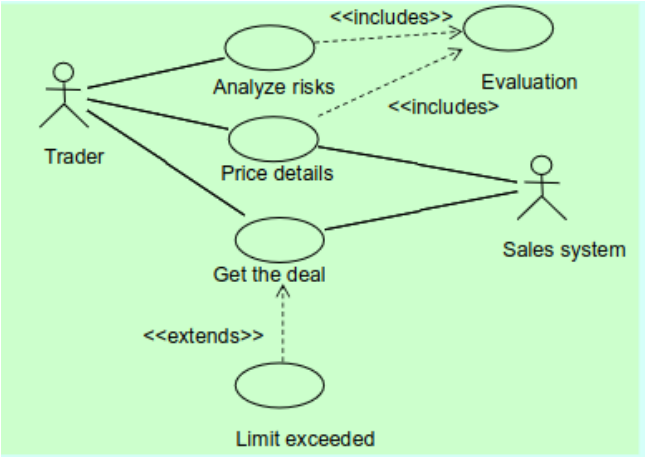



**Cosa e' un Use case?** Interazione tipica tra attori e il sitema, processa a soddisfare il bisogno degli utenti, descrivendo uno scenario nel quale il sistema e' utilizzato.

- Granularita': grande o piccola
- Spresso: funzioni user-visible
- Sempre: puntato a un obbiettivo discreto
- Sempre: descrivono una funziona requisito esterna

**Quando e come?**

- Contest capture: prima cosa da fare
- Use case: tutte le cose discrete che il clcinete vuole fare con il sistema (nome, descrizione, dettagli futuri, ...)



**Attori** Ruoli che gli utenti fanno nel sisytema, gli attori applicano gli use case. Gli attori non devono per forza essere umani, e possono ricevere ricompensa in caso di partecipazione.

**Relazioni estese** Un use case puo' estendere un altro use case, aggiungendo funzionalita' opzionali.

**Relazioni include** Un use case puo' includere un altro o piu' use case.

| **Estese** | **Incluse** |
|---|---|
| lo stesso attore esegue lo use case e tutte le estensioni | spesso nessun attore è associato allo use case comune |
| l'attore è collegato allo use case "base" | sono possibili attori diversi per gli use case "caller" |

**Descrizione testuale** Step by step delle interazione tra attori e use case (corto, preciso)

### 2.2.5 Analisi OO

L'OOA possiamo definirla come:

- estrarre le classi candidate
- stabilire le relazioni di base tra le classi
- definire una gerarchia di classi
- identificare gli attributi di ogni classe
- specificare i metodi che operano sugli attributi
- indicare come classi/oggetti sono in relazione
- costruire un modello comportamentale
- iterare sui primi cinque passi

**Diagrama classe** Ovviamento è centrale nei modelli OO, mostrano la struttura statica del sistema (tipi degli oggetti, relazioni come associazione, sottotipi e dipendenze)

**3 punti di vista**

- Concettuale (OOA)
  - Mostra i concetti del dominio
  - Indipendente dall'implementazione
- Specifica (OOD)
  - Struttura generale del sistema in esecuzione

  - Interfacce del software (tipi)
- Implementazione (OOP)
  - Dettagli dell'implementazione
  - Molto spesso l'unico utilizzato

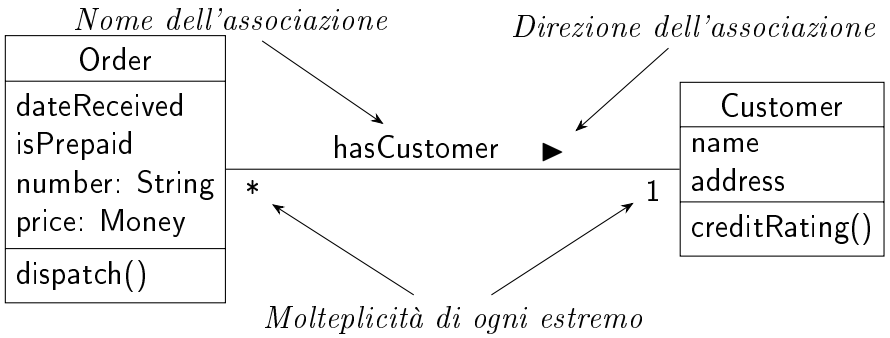
Una **classe** definisce un insieme di oggetti, e definisce *nome, attributi e operazioni.*

**Classi vs tipi**

| **Tipo** | **Classe** |
|---|---|
| protocollo compreso da un oggetto | costrutto orientato all'implementazione |
| insieme delle operazioni utilizzate | implementa uno o più tipi |

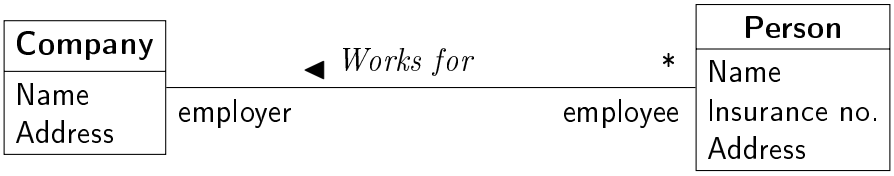
Definiamo **associazione** una relazione tra instanze di una classe.

**Classi e diagrammi** One classe può far parte di diversi diagrammi. Questi dovrebbero illustrale specifici aspetti, quindi omettere attributi/operazioni irrilevanti.

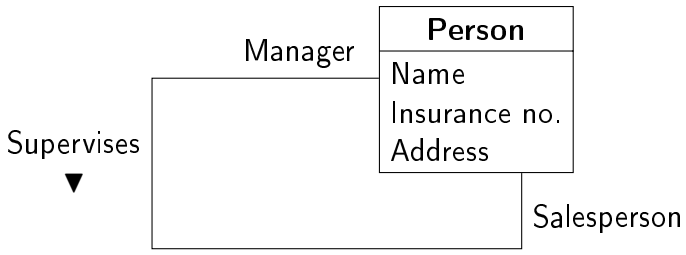



Un ordine viene da un cliente, ma un cliente può avere molti ordini.

**Ruoli** Una associazione ha 2 ruoli, un ruole è una direzione sulla associazione. Un ruolo ha una etichetta esplicita o implicita dentro il nome della classe target.
Un ruolo indica una estremità della associazione:



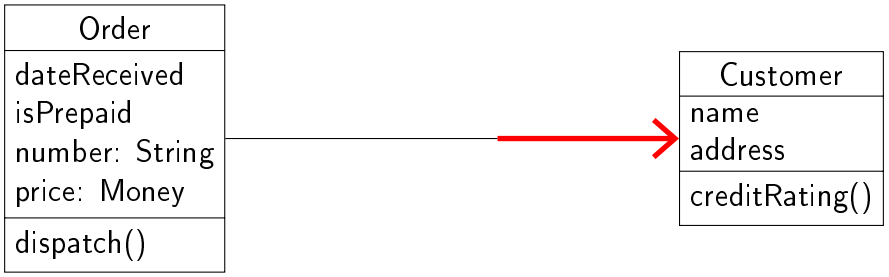

Il nome del ruolo è obbligatorio per una associazione tra due oggetti della stessa classe



**Molteplicità** Indicano quanti oggetti possono partecipare in una

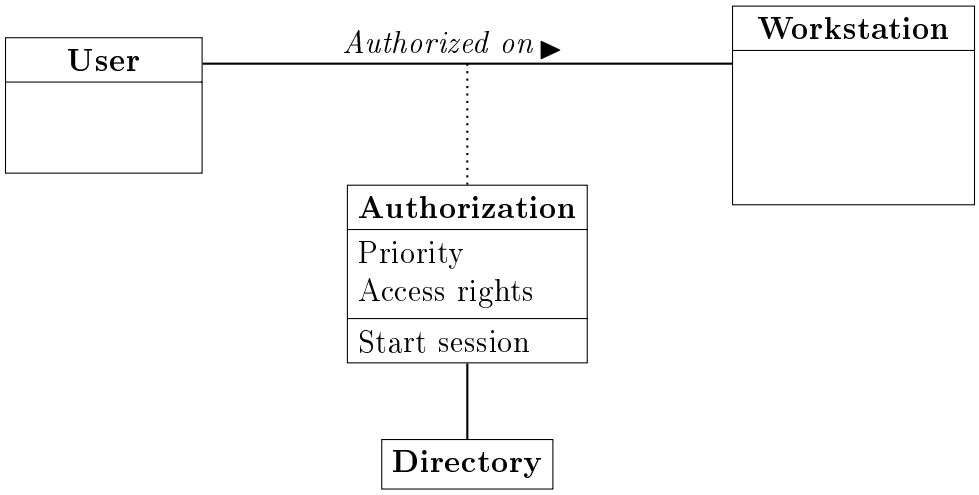



**Navigabilità** indicata dalla freccia

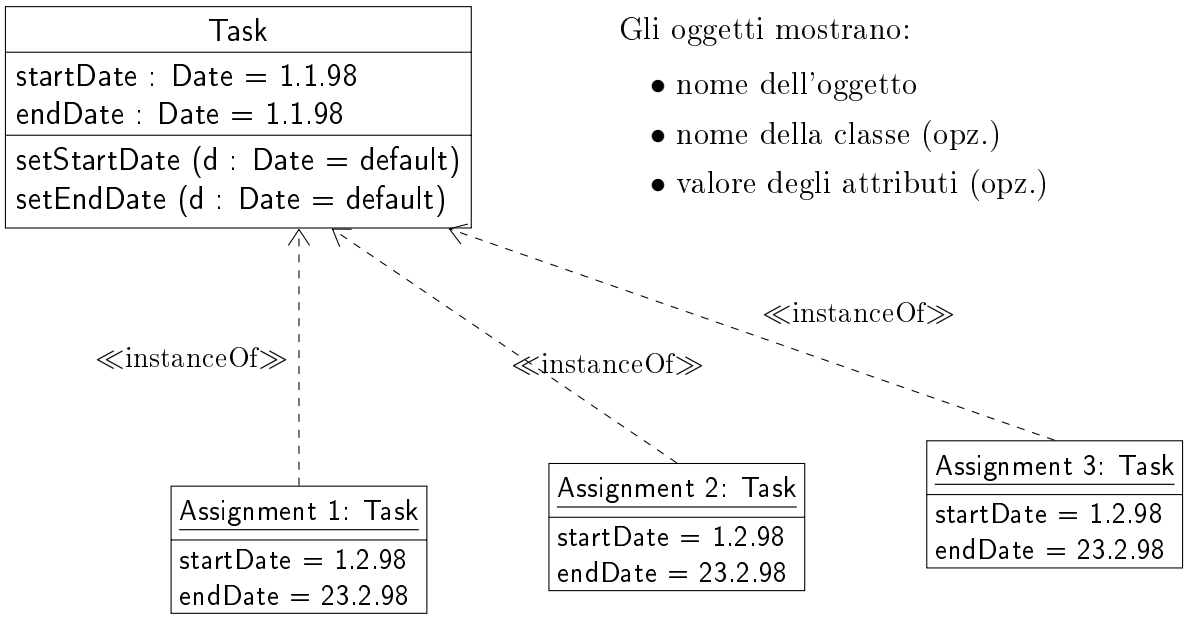



Dall'ordine dobbiamo poter capire chi è il cliente, il cliente però non sa tutti gli ordini.

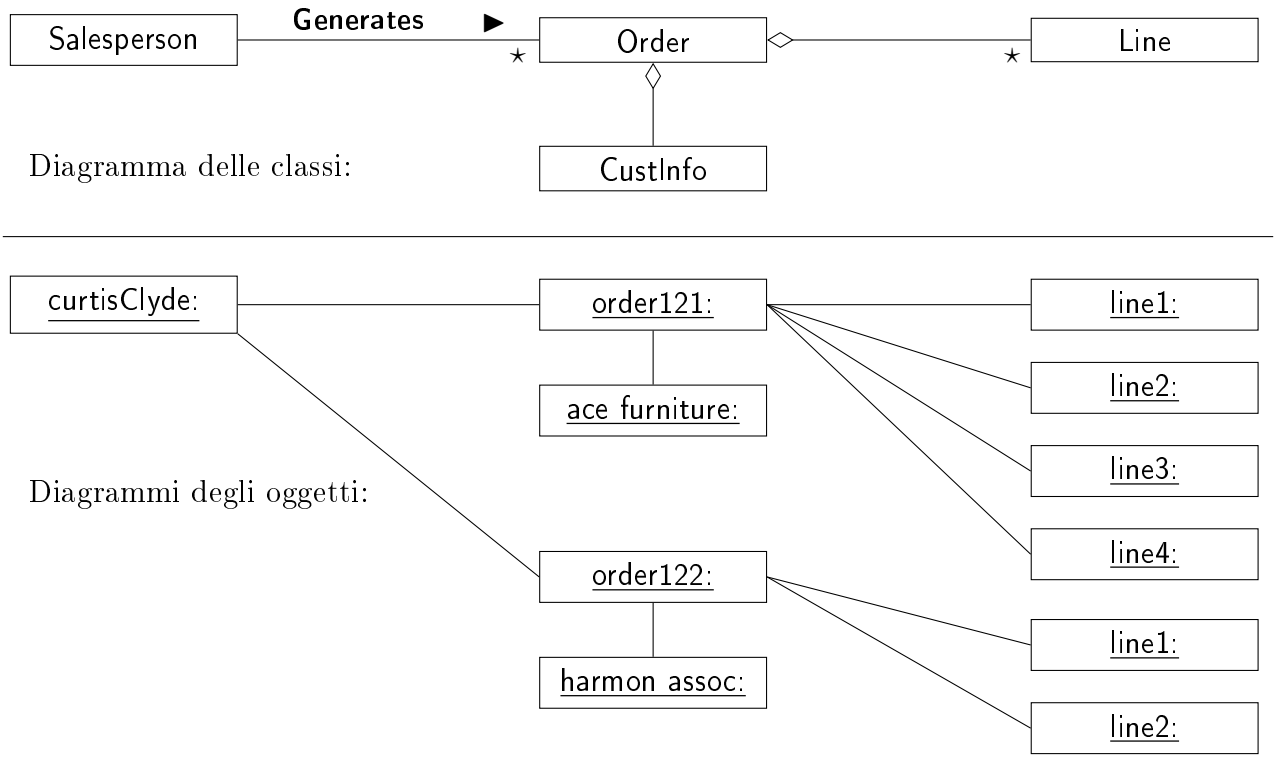
**Associazione bi-direzionale** Navigabilità in entrambe le direzioni.



**Classi e oggetti** Come abbiamo detto prima le classi definiscono una struttura a gruppo su oggetti, definendo nomi, attriuti e operazioni




**Esempio: Classe vs Oggetti**




**Attributi**

- **Conceptual**: indicates that customer have names.
- **Specification**: customer can tell you the name and set it.
- **Implementation**: an instance variable is available.

### Differenze attributi vs. associazione

- **Prospettiva concettuale**: non c'è molta differenza! Gli attributi sono a valore singolo (0..1).
- **Prospettiva di specifica/implementazione**:
  - navigabilità dal tipo all'attributo;
  - l'attributo memorizza *valori*, NON riferimenti: non c'è condivisione dei valori degli attributi tra istanze diverse!
  - spesso memorizza oggetti semplici: numeri, stringhe, date, oggetti *Money*.

### Operazioni

- Processi che una classe sa eseguire; corrispondono ai messaggi della classe.
- **Livello concettuale**: le responsabilità principali.
- **Livello di specifica**: i messaggi pubblici, cioè l'interfaccia della classe.
- Normalmente non si mostrano le operazioni che manipolano gli attributi.

### Sintassi UML per le operazioni

*visibilità nome (lista parametri)* : *tipo di ritorno*

es. `+ assignAgent (a : Agent) : Boolean`

- **visibilità**: **pubblica** (+), **protetta** (#), **privata** (-);
  - l'interpretazione dipende dal linguaggio;
  - non serve a livello concettuale.
- **nome**: stringa;
- **lista parametri**: argomenti (stessa sintassi degli attributi);
- **tipo di ritorno**: specifica dipendente dal linguaggio.

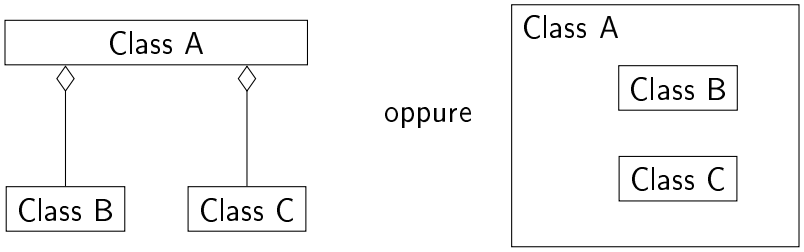
### Tipi di operazioni

- **Query**: restituisce un valore senza modificare lo stato interno della classe.
- **Modifier**: modifica lo stato interno.
- Le query possono essere eseguite in qualsiasi ordine.
- Messaggi di *getting* e *setting*:
  - getting: query;
  - setting: modifier.

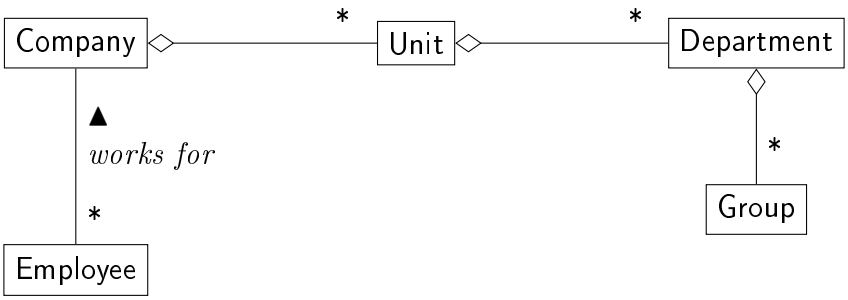
**Aggregazione**

- Forma speciale di associazione.
- I componenti sono parti dell'oggetto aggregato: un'auto ha un motore e delle ruote come sue parti.
- L'aggregazione è transitiva.
- Esempi tipici:
  - esplosione delle parti (*parts explosion*);
  - struttura organizzativa di un'azienda.

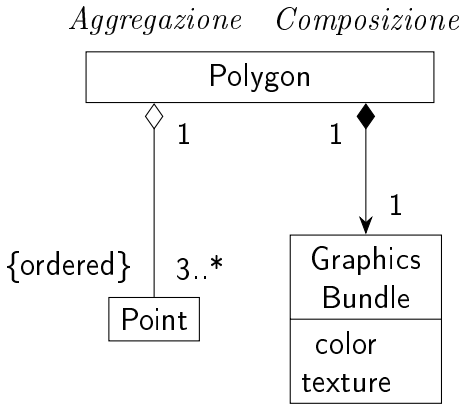
**Notazione per l'aggregazione** Un rombo vuoto dal lato dell'aggregato, oppure le parti disegnate dentro la classe aggregata:




**Esempio di aggregazione**




**Aggregazione e composizione**




**Composizione**:

- i componenti appartengono a un solo intero;

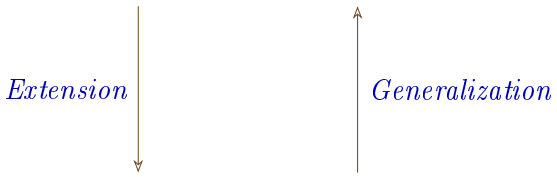
- le parti vivono e muoiono con l'intero:
  - cancellazione a cascata;
  - necessaria anche per le associazioni 1..1;
- i giocatori possono essere aggregati per le Fiamme, MA non vengono eliminati quando le Fiamme scompaiono.

**Come riconoscere un'aggregazione** Associazione o aggregazione?

- la descrizione «parte di» è corretta?
- un'operazione sull'intero riguarda anche le parti?

**Generalizzazione vs. estensione**

- Car
  - Truck
  - Bus
  - Station wagon




- Molte cose in comune.
- Alcune differenze.

**Concetto di generalizzazione**

- Classe: definisce implicitamente un insieme di oggetti.
  - aCar ∈ Car = insieme di tutte le auto (*istanziazione*);
- Generalizzazione: relazione di sottoinsieme.
  - Truck ⊆ Car (*generalizzazione*).

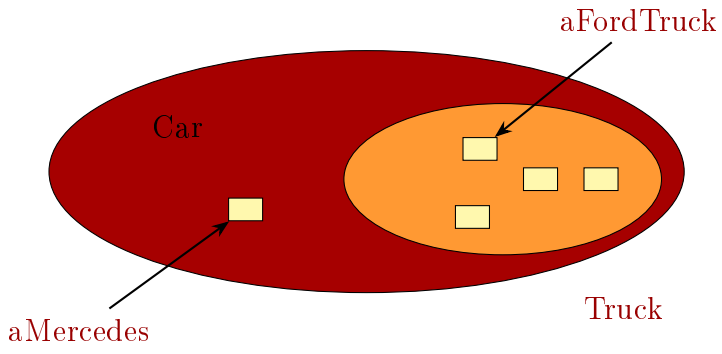
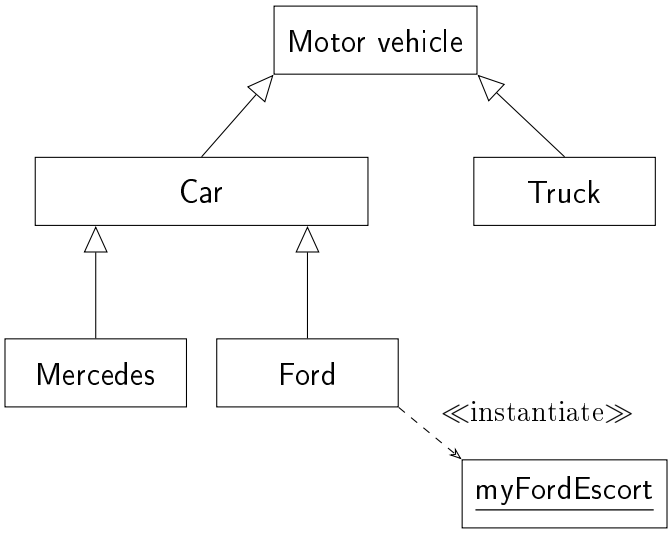


**Diagramma delle classi con ereditarietà**




**Come definire le classi (rivisto)**

- Cercare i nomi (*nouns*) nei casi d'uso.
- Definire una classe per ogni nome (e aggiungerne altre).
- Documentare l'insieme di regole che determina l'insieme di oggetti appartenenti alla classe.
- Aggiungere associazioni per modellare le relazioni.
- Pensare alla relazione di sottoinsieme per costruire le generalizzazioni...

**A quale classe appartiene un oggetto?**

- Definizione dell'appartenenza a una classe:
  - **implicita, tramite regole**:
    - le regole definiscono la condizione per essere membro della classe;
    - valori degli attributi disponibili $\Rightarrow$ la classe può essere determinata;
    - logica terminologica dell'IA (*subsumption*, *classifier*);
  - **esplicita, per enumerazione**:
    - l'istanziazione definisce l'appartenenza alla classe;
    - problema: vietare le operazioni che violano le restrizioni.

**Cambiare classe**

- UML dà a un oggetto la possibilità di cambiare dinamicamente la propria classe (usando lo stereotipo *type*).
- In C++ e Java ciò si ottiene avendo una classe base comune e cambiando poi gli oggetti puntati/referenziati tramite costruttori adeguati.

### Generalizzazione: estensione e restrizione

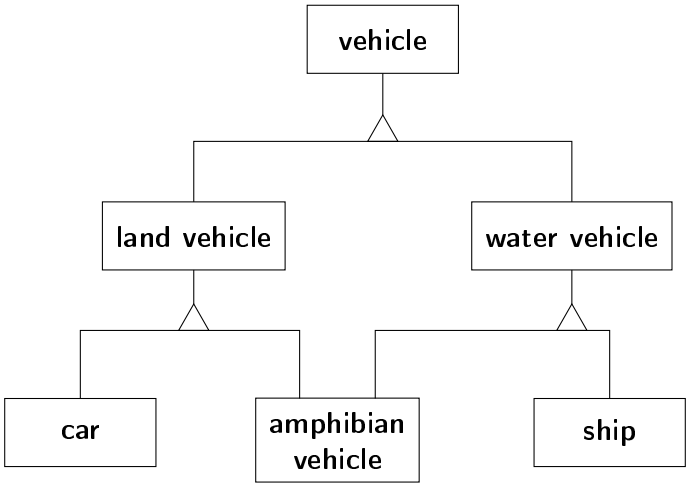
- Attributi e operazioni di una classe antenata sono ereditati dalla sottoclasse.
- **Estensione**: aggiunta di nuovi attributi o operazioni.
- **Restrizione**: restrizioni aggiuntive sugli attributi dell'antenata.
  - cerchio = ellisse con gli assi della stessa lunghezza;
  - attenzione: la modifica arbitraria della dimensione di un asse del cerchio può violare la restrizione.

### Prospettive

- **Conceptual**: relazione di sottoinsieme.
- **Specification**: il sottotipo è conforme all'interfaccia del supertipo.
- **Implementation**: ereditarietà di implementazione, *subclassing*.

**ATTENZIONE**: non creare sottoclassi quando il livello concettuale non lo supporta; usare invece l'aggregazione. (Uno *stack* **non** è una lista con qualche *overriding*!)

**Ereditarietà multipla** La classe eredita caratteristiche da più superclassi.




### Ereditarietà virtuale

- `class ItemToSell { double price; };`
- `class EducationalTool : virtual public ItemToSell { };`
- `class Book : virtual public ItemToSell { };`
- `class TextBook : public EducationalTool, public Book { };`

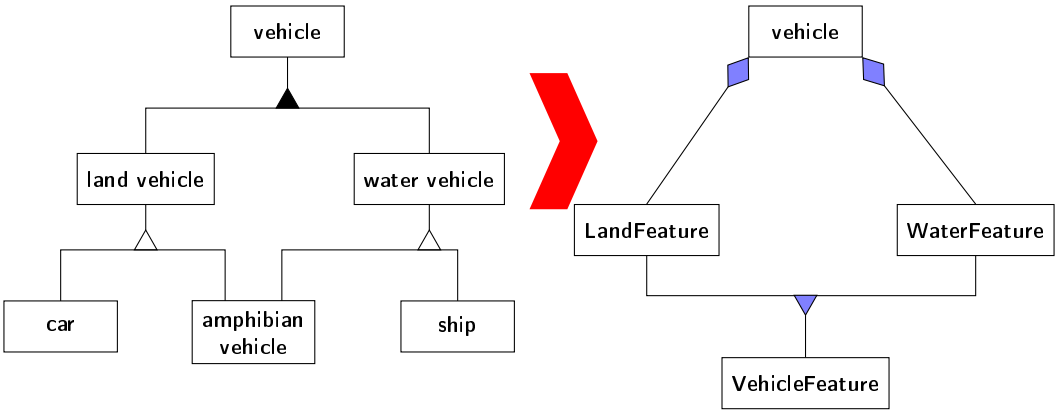
### Discussione sull'ereditarietà multipla

- **Vantaggi**:
  - più vicina al modo di pensare umano;
  - maggiore flessibilità nella specifica delle classi;
  - maggiori possibilità di riuso.
- **Svantaggi**:
  - perdita di chiarezza ⇒ quale metodo viene eseguito?
  - implementazione più complicata;
  - è necessaria una risoluzione dei conflitti per le caratteristiche ereditate più volte.

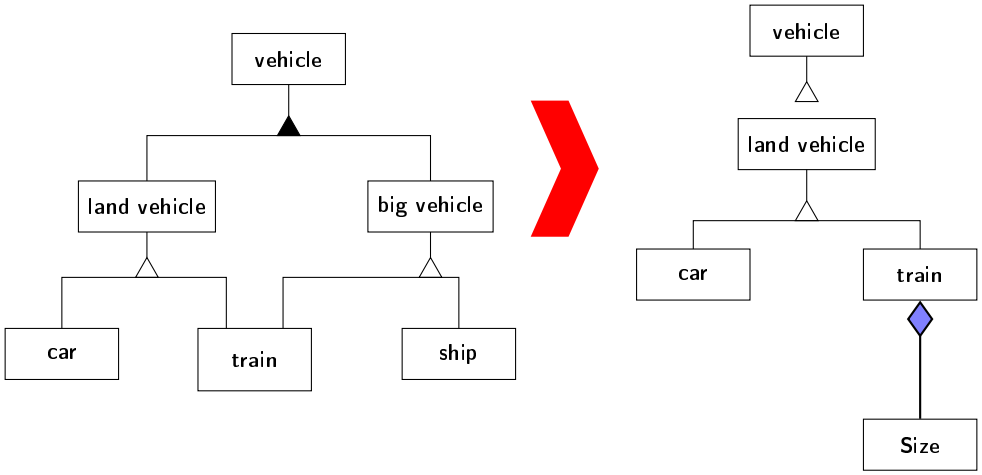
### Evitare l'ereditarietà multipla

- Fondamentalmente: una questione di implementazione.
- Spesso il modo più semplice è ristrutturare il modello.
- Tecniche di ristrutturazione:
  - delegazione e aggregazione;
  - ereditarietà basata sulla caratteristica più importante e delegazione del resto;
  - generalizzazione basata su dimensioni diverse.

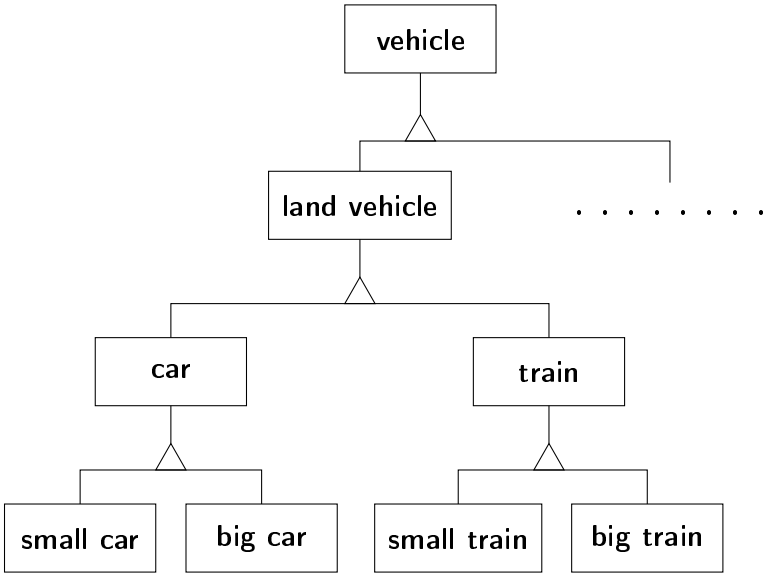
### Delegazione e aggregazione




### Caratteristica più importante e aggregazione



### Generalizzazione basata su dimensioni diverse




### Quando usare i diagrammi delle classi

- I diagrammi delle classi sono la spina dorsale degli approcci di sviluppo a oggetti.
- Non usare tutte le notazioni:
  - partire dalle cose semplici.
- Tenere conto della prospettiva:
  - non troppi dettagli in analisi;
  - la specifica è spesso meglio dell'implementazione.
- Concentrarsi sulle aree chiave:
  - meglio pochi diagrammi aggiornati che molti modelli obsoleti.

### Creare un diagramma delle classi

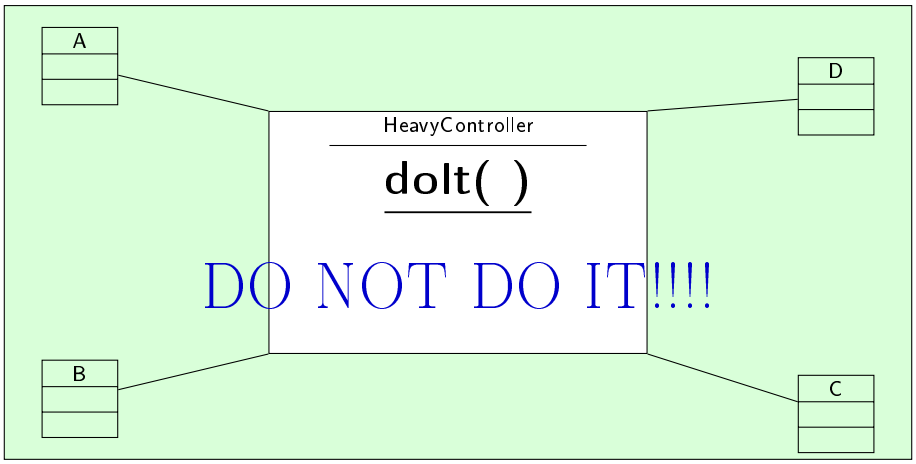
- Partire in modo semplice:
  - classi principali e associazioni ovvie.
- Poi aggiungere:
  - attributi;
  - molteplicità;
  - operazioni.

**Regole pratiche**

- Una classe può far parte di più diagrammi.
- I diagrammi devono illustrare aspetti specifici:
  - non troppe classi;
  - non troppe associazioni;
  - nascondere attributi/operazioni irrilevanti.
- Servono più iterazioni per creare un diagramma.

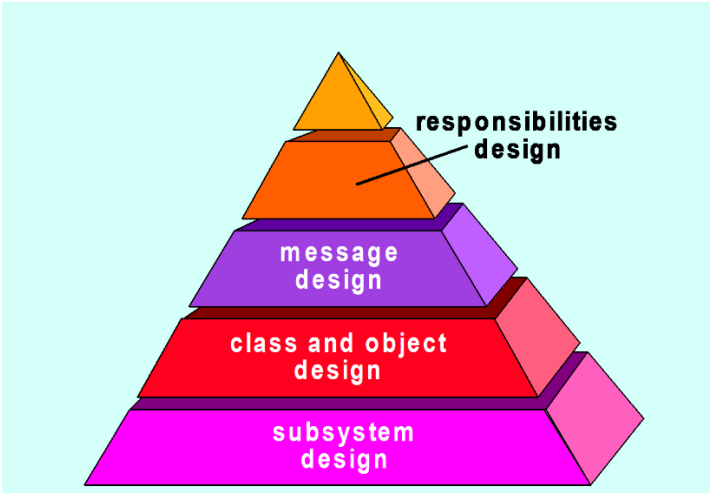
**Evitare le classi «pesanti»**

- Il controller fa tutto.
- Le altre classi si limitano all'incapsulamento dei dati.




## 2.3 OO Design in UML

### 2.3.1 L'idea della progettazione a oggetti (OOD)



### 2.3.2 Problematiche di progetto

Un metodo di progettazione è giudicato modulare in base a cinque criteri:

- **Decomponibilità** — la facilità con cui il metodo aiuta il progettista a scomporre un problema grande in sottoproblemi più semplici da risolvere.
- **Componibilità** — il grado in cui il metodo garantisce che i componenti (moduli) del programma, una volta progettati e costruiti, possano essere riutilizzati per creare altri sistemi.
- **Comprensibilità** — la facilità con cui un componente può essere capito senza far riferimento ad altre informazioni o ad altri moduli.
- **Continuità** — la capacità di fare piccole modifiche al programma facendo sì che si riflettano in modifiche in uno solo, o in pochissimi, moduli.
- **Protezione** — caratteristica architetturale che riduce la propagazione degli effetti collaterali quando si verifica un errore in un dato modulo.

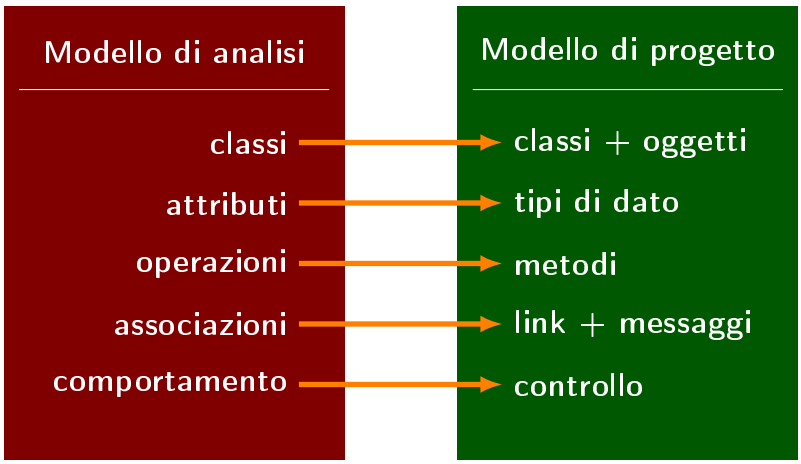
#### Ingredienti per OOD

- **Componente del dominio del problema** — i sottosistemi responsabili di implementare direttamente i requisiti del cliente.
- **Componente di interazione umana** — i sottosistemi che implementano l'interfaccia utente (inclusi i sottosistemi GUI riutilizzabili).
- **Componente di gestione dei task** — i sottosistemi responsabili di controllare e coordinare i task concorrenti, che possono essere racchiusi in un sottosistema o distribuiti tra sottosistemi diversi.
- **Componente di gestione dei dati** — il sottosistema responsabile della memorizzazione e del recupero degli oggetti.

### 2.3.3 OOA vs. OOD

- **Livello di dettaglio**:
  - i nomi sono fissati;
  - le firme dei messaggi sono fissate;
  - molteplicità e sua realizzazione;
  - visibilità;
  - algoritmi dei metodi;
  - diagrammi di sequenza/collaborazione più dettagliati;
  - ...
- **Notazioni di diagramma aggiuntive**.

### Relazione tra OOA e OOD




In OOD si usano ancora i diagrammi delle classi, ma raffinati per adattarsi al progetto del sistema. Oltre ai diagrammi delle classi ci sono diversi altri diagrammi:

**Diagrammi strutturali**

- ✓ diagrammi degli oggetti;
- ✓ diagrammi di deployment.

**Diagrammi comportamentali**

- ✓ diagrammi di sequenza;
- ✓ diagrammi di collaborazione;
- ✓ diagrammi di stato (statechart);
- ✓ diagrammi di attività.

### Diagrammi strutturali per OOD in UML

- **Diagrammi delle classi**: la struttura è la stessa dell'OOA.
- **Diagrammi degli oggetti**:
  - trattano oggetti, cioè istanze delle classi;
  - sono del tutto omomorfi ai diagrammi delle classi;
  - per questo non li analizzeremo a fondo.

### Diagrammi comportamentali per OOD in UML

- **Diagrammi di stato (statechart)**: descrivono l'evoluzione degli stati di un qualsiasi classificatore; usati comunemente per gli oggetti.
- **Diagrammi di attività**: descrivono l'evoluzione delle attività nel sistema.
- **Diagrammi di sequenza**: descrivono le interazioni tra oggetti secondo l'ordine temporale.
- **Diagrammi di collaborazione**: descrivono le interazioni tra oggetti secondo la loro organizzazione.

Diagrammi di sequenza e di collaborazione sono detti *diagrammi di interazione*.

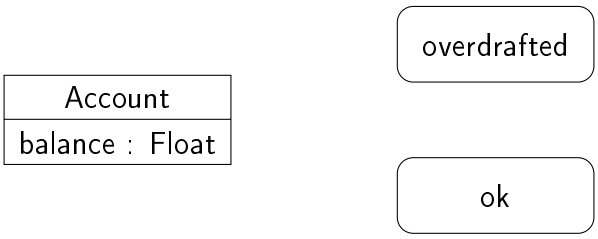
### 2.3.4 Diagrammi di stato (Statechart)

**Diagrammi di stato vs. diagrammi di interazione**

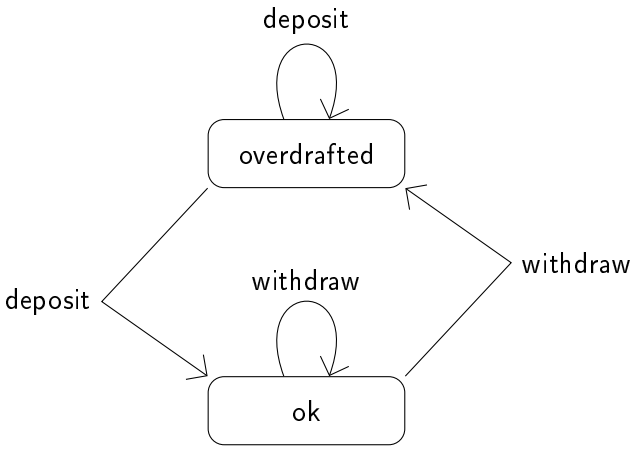
- **Diagrammi di interazione**: mostrano come gli oggetti interagiscono tra loro.
- **Diagrammi di stato**: mostrano il comportamento di *un solo* oggetto:
  - come cambia il suo stato in base ai messaggi che riceve;
  - focalizzati su un ambito ristretto, a grana fine.
- **Altri nomi**: diagrammi di transizione di stato (*state transition*), diagrammi degli stati (*state diagrams*), diagrammi di Harel.

**Stati di un oggetto**

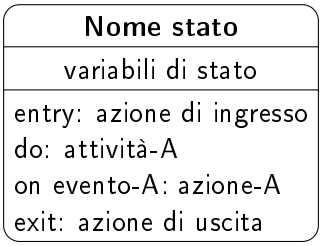
- **Stato** = insieme dei valori che descrivono un oggetto in uno specifico istante di tempo.
- Lo stato è determinato in base ai valori degli attributi.




**Cambi di stato** Gli stati possono cambiare quando si verifica un *evento*. Gli eventi sono i messaggi ricevuti; un evento può cambiare lo stato oppure no.




**Notazione: lo stato** Uno stato si disegna come un rettangolo arrotondato con il nome ed eventualmente le variabili di stato e le azioni/attività interne:



- **Azione**: avviene rapidamente e non è interrompibile (cosa significhi «rapidamente» dipende dal contesto).
- **Attività**: può durare a lungo e può essere interrotta.
- `entry` — azione eseguita all'ingresso nello stato;
- `do` — attività in corso finché si resta nello stato (es. mostrare una finestra);
- `on` — azione eseguita in risposta a uno specifico evento;
- `exit` — azione eseguita all'uscita dallo stato.

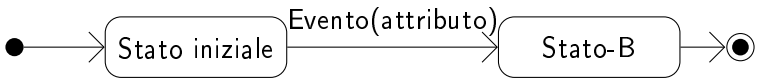
**Notazione: la transizione** Una transizione è una freccia tra due stati etichettata con

```
Evento(argomenti) [condizione] / azione
```

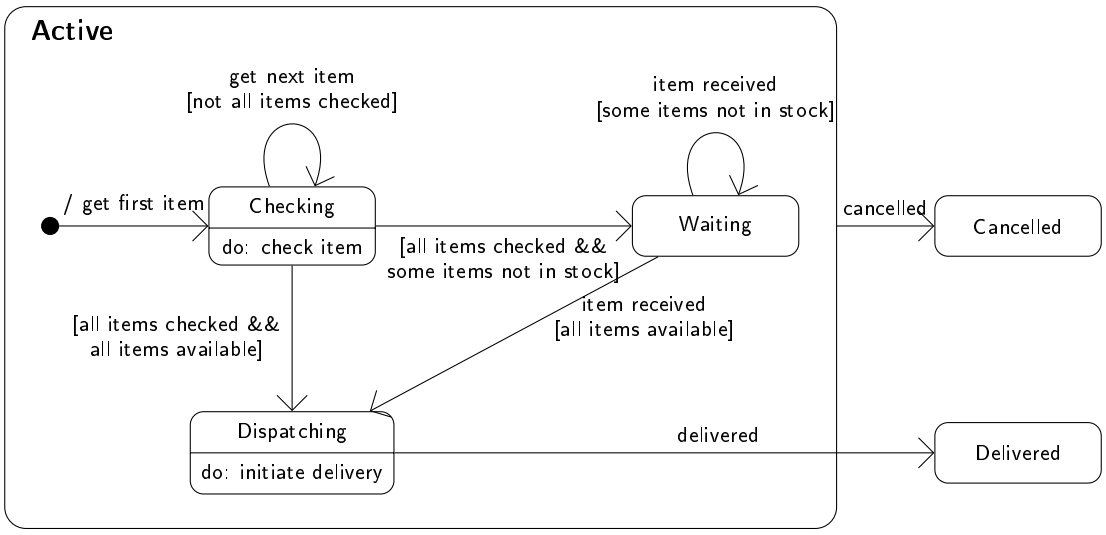
- **Evento**: l'invio di un messaggio.
- **Condizione di guardia**: la transizione avviene solo se la guardia è vera; le guardie delle transizioni che escono da uno stesso stato devono essere mutuamente esclusive.
- **Azione**: processo considerato rapido e non interrompibile.

Ogni parte dell'etichetta può essere omessa.

Lo **stato iniziale** si indica con un pallino pieno, lo **stato finale** con un pallino pieno cerchiato:




**Superstati e sottostati** Se più stati condividono la stessa transizione uscente (es. poter annullare un ordine in qualsiasi momento), invece di disegnare una transizione da ogni stato si raggruppano in un **superstato** e si disegna una sola transizione che esce dal superstato. Esempio, gestione di un ordine:




L'evento *cancelled* porta in Cancelled da qualunque sottostato di Active.

**Note**

- I diagrammi di stato *non* devono per forza riferirsi a classi o oggetti: possono descrivere anche sottosistemi ecc.; l'uso più diffuso resta comunque quello sugli oggetti.
- Argomenti non trattati: stati annidati vs. stati concorrenti, stati *history*, transizioni interne.

### 2.3.5 Diagrammi di attività

I diagrammi di attività si usano per descrivere:

- *workflow* (flussi di lavoro);
- elaborazioni parallele.

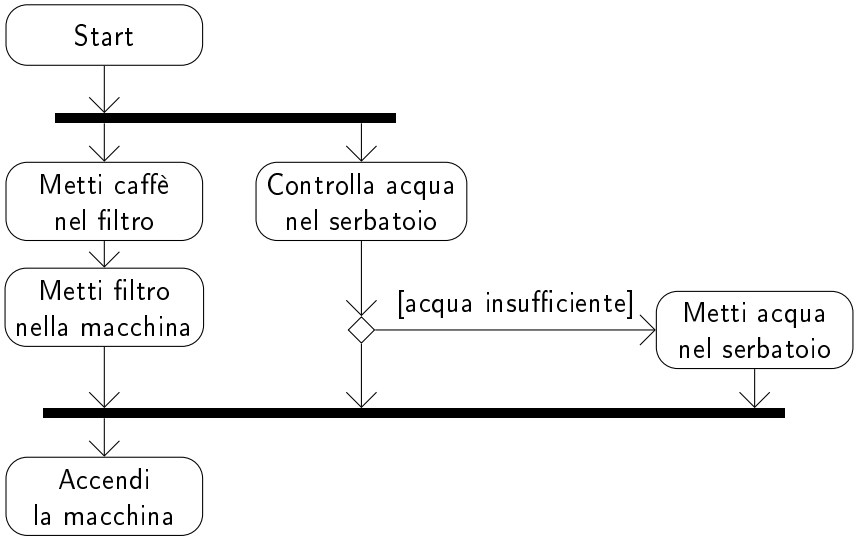
Un'**attività** può essere intesa a livello *concettuale* (un compito da svolgere) oppure a livello di *specifica/implementazione* (un metodo di una classe). Sono simili alle reti di Petri.

**Struttura**

- **Barra di sincronizzazione**: una barra spessa.
  - **Fork** (una freccia entrante, più uscenti): le attività successive possono essere svolte in parallelo, in qualsiasi ordine;
  - **Join** (più frecce entranti): si prosegue solo quando *tutti* i predecessori sono terminati; può avere una `[condizione di sincronizzazione]`.
- **Branch**: un rombo da cui escono più frecce, ciascuna con una **guardia** `[condizione]`.
- Inizio e fine si indicano come negli statechart (pallino pieno, pallino cerchiato).

Un diagramma di attività mostra quindi un *ordine parziale* tra le attività.

Esempio, preparare il caffè:




Dopo il fork le due attività procedono in parallelo; il branch fa eseguire «Metti acqua» solo se la guardia è vera; il join attende tutti i rami prima di accendere la macchina.

Argomenti non trattati: attività vs. azioni, *swimlane* e flussi di oggetti, cambi di stato di un oggetto.

### 2.3.6 Diagrammi di sequenza

I diagrammi di sequenza e di collaborazione sono **diagrammi di interazione**: si concentrano sulle entità «reali», gli oggetti, e descrivono in dettaglio come comunicano tra loro. Offrono due viste:

- vista basata sul **tempo** → diagrammi di sequenza;
- vista basata sull'**organizzazione** → diagrammi di collaborazione.

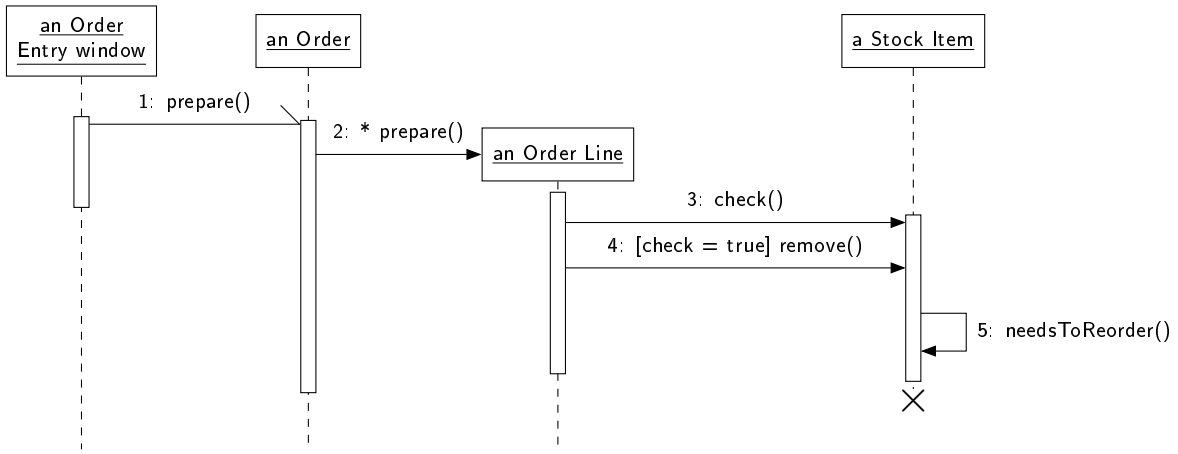
Un diagramma di sequenza mostra le interazioni tra oggetti ordinate nel tempo, concentrandosi sugli oggetti (e classi) e sui messaggi scambiati per realizzare la funzionalità di uno scenario. Gli oggetti sono disposti su una linea *orizzontale*, gli eventi lungo una linea del tempo *verticale* (*lifeline*). I messaggi vanno dal *client* al *supplier*.

#### Contenuto

- **Oggetti**, che si scambiano messaggi.
- **Messaggi**:
  - *sincroni* («call event»): freccia piena;
  - *asincroni* («signal»): mezza freccia;
  - messaggi «create» e «destroy».

Un messaggio **asincrono** non blocca il chiamante e può: creare un nuovo thread, creare un nuovo oggetto, comunicare con un thread già in esecuzione.

#### Notazione




- **Attivazione**: il rettangolo sulla lifeline, periodo in cui l'oggetto sta eseguendo un'operazione.
- **Messaggio asincrono**: mezza freccia (1).
- **Iterazione**: `*` davanti al messaggio, inviato più volte (2).
- **Creazione**: il messaggio punta al riquadro dell'oggetto, disegnato all'altezza in cui viene creato (2).
- **Condizione**: `[condizione]` davanti al messaggio, inviato solo se vera (4).
- **Autodelega**: messaggio che un oggetto invia a sé stesso (5).
- **Distruzione**: una × alla fine della lifeline.

**Complessità** I diagrammi servono a rendere le cose chiare: **KISS** (*keep it small and simple*). Logica condizionale semplice si aggiunge al diagramma; se è complessa si disegnano diagrammi separati.

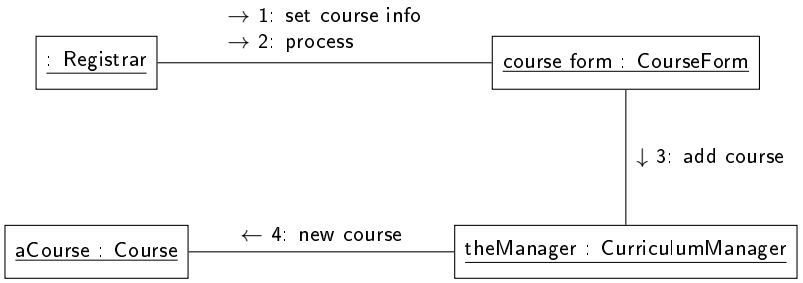
**Confini (boundary)** Un *boundary* modella la comunicazione tra il sistema e il mondo esterno (es. interfaccia utente o altro sistema). Può essere utile mostrare le classi boundary nei diagrammi di interazione per catturare i requisiti di interfaccia, *senza* mostrare come l'interfaccia sarà implementata.

### 2.3.7 Diagrammi di collaborazione

Mostrano come gli oggetti interagiscono rispetto alle unità organizzative (i *boundary*). Il contenuto è quasi lo stesso dei diagrammi di sequenza (oggetti, messaggi sincroni/asincroni, «create» e «destroy», cicli), ma l'ordine dei messaggi è dato dalla **numerazione**:

- piatta: 1, 2, 3, 4, ...
- annidata: 1, 1.1, 1.2, 2, 2.1, 2.1.1, ... — mostra quale operazione chiama quale altra.

Esempio (registrazione di un corso):




Gli oggetti sono collegati da *link*; i messaggi sono scritti lungo il link con una freccia che ne indica il verso.

#### Sequenza vs. collaborazione

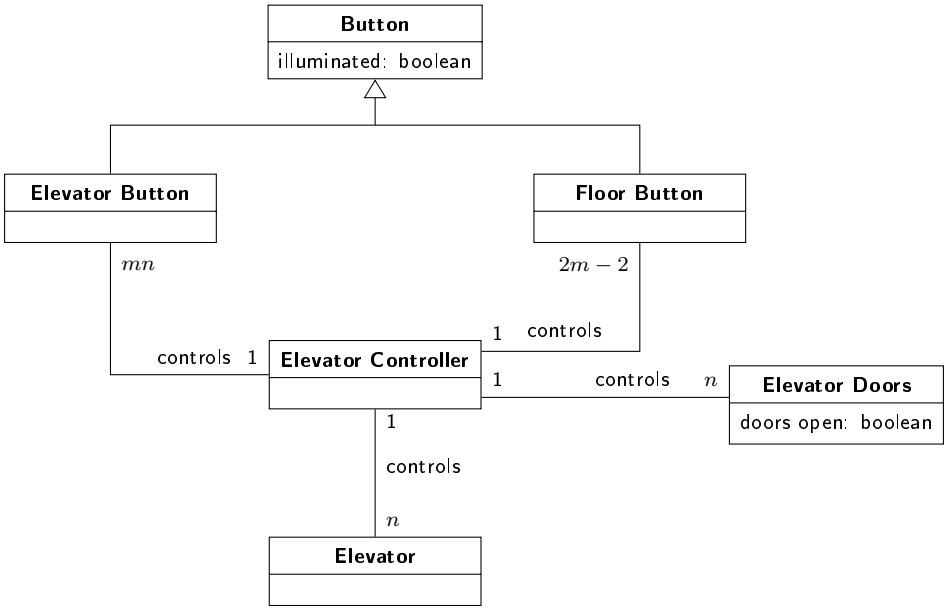
- Il flusso nel **tempo** si vede meglio nei diagrammi di sequenza; nei diagrammi di collaborazione la sequenza dei messaggi è più difficile da seguire.
- Il flusso di controllo rispetto all'**organizzazione** si vede meglio nei diagrammi di collaborazione, il cui layout può mostrare le connessioni statiche tra oggetti.
- Un controllo complesso è comunque difficile da esprimere in entrambi; i diagrammi di collaborazione possono diventare *molto* complessi.

### 2.3.8 Esempio: mettere insieme tutti i pezzi

Un esempio completo che usa tutti i diagrammi visti: un sistema di $n$ ascensori per un edificio di $m$ piani.

**Caso d'uso** L'utente preme un pulsante dell'ascensore, preme un pulsante del piano, entra ed esce attraverso una porta. Dai sostantivi emergono gli oggetti: pulsanti, ascensore, porte.

**Diagramma delle classi**




Elevator Button e Floor Button specializzano Button. Ci sono $mn$ pulsanti nelle cabine ($m$ per ognuno degli $n$ ascensori) e $2m-2$ pulsanti ai piani (su e giù, tranne al primo e all'ultimo piano). Il controller gestisce tutti i pulsanti, le $n$ porte e gli $n$ ascensori.

**Diagramma di stato del controller** Il controller è un ciclo: se un pulsante viene premuto e non è acceso, elabora la richiesta (`do/update request`, `do/turn on button`); mentre l'ascensore si muove verso il piano $f$ controlla se qualcuno ha chiesto di fermarsi lì: se no continua a muoversi, se sì si ferma, apre le porte, aggiorna le richieste e spegne i pulsanti relativi; da fermo, se ci sono altre richieste chiude le porte e serve la successiva, altrimenti entra in attesa chiudendo le porte dopo un timeout.

**Scenario: sequenza e collaborazione** Lo stesso scenario si rappresenta sia con un diagramma di sequenza sia con uno di collaborazione (oggetti: User A, Floor Button, Elevator Button, Elevator Controller, Elevator, Elevator Doors):

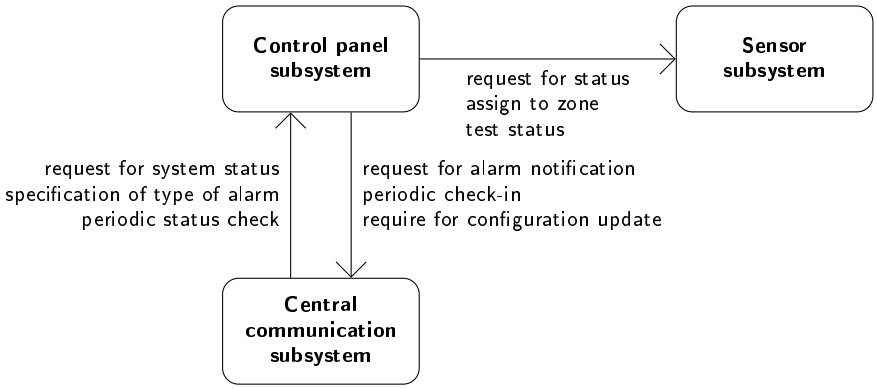
1. l'utente preme il pulsante del piano;
2. il pulsante del piano si accende;
3. $*$ l'ascensore sale di un piano (ripetuto);
4. il pulsante del piano si spegne;
5. le porte si aprono;
6. l'utente preme il pulsante in cabina;
7. il pulsante in cabina si accende;
8. le porte si chiudono;
9. $*$ l'ascensore sale di un piano;
10. il pulsante in cabina si spegne;
11. le porte si aprono;

12. parte il timer;
13. le porte si chiudono;
14. * l'ascensore si muove di un piano.

Il movimento può essere verso l'alto o verso il basso. Nel diagramma di sequenza si legge bene l'ordine temporale; in quello di collaborazione si vede che tutti i messaggi passano dall'Elevator Controller, che è il centro dell'organizzazione.

### 2.3.9 Sistemi e sottosistemi

Un sistema grande si scompone in **sottosistemi** che comunicano tra loro. Esempio (sistema di sicurezza domestica):




**Come scomporre un sistema?**

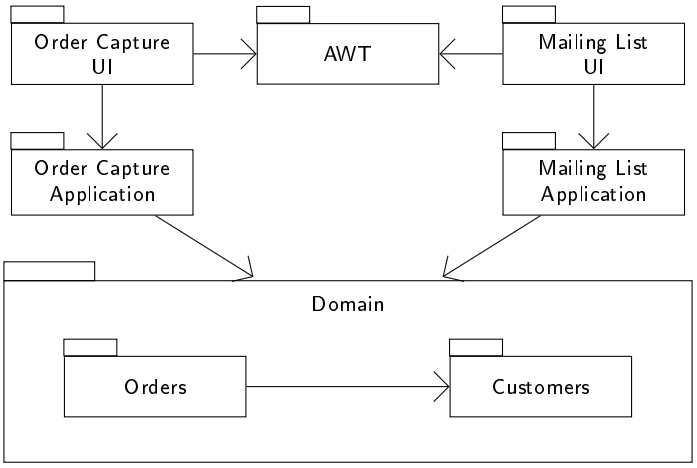
- Principio romano: *divide et impera* — dividere un sistema grande in parti gestibili.
- Metodi strutturati: **decomposizione funzionale**.
- OO: **raggruppare le classi** in unità di livello più alto, cioè *package* e *componenti*.

### 2.3.10 Diagrammi dei package

Un diagramma dei package mostra i package e le **dipendenze** tra loro. Esiste una dipendenza tra due elementi se modificare la definizione di uno può causare modifiche all'altro.

Obiettivo (e arte) della progettazione su larga scala: **minimizzare le dipendenze**, così da limitare gli effetti delle modifiche.

**Package annidati** Esempio, sistema per inviare e gestire ordini:



Una dipendenza verso un package che ne contiene altri ha due interpretazioni:

- **trasparente**: è visibile tutto il contenuto dei package contenuti;
- **opaca**: sono visibili solo le classi del package contenitore.

#### Limitare la complessità dell'interfaccia di un package

- dare a tutte le classi del package visibilità di *package*;
- definire una classe pubblica che fornisce il comportamento pubblico del package;
- delegare le operazioni pubbliche alle classi opportune all'interno del package.

È il pattern **Facade** [Gamma et al. 1994].

#### Regole pratiche

- evitare cicli nella struttura delle dipendenze;
- troppe dipendenze: provare a fare refactoring del sistema;
- usare i package quando il diagramma delle classi del sistema non è leggibile su un singolo foglio A4.

#### Package vs. diagrammi

- I **package** rappresentano divisioni *fisiche* dello sviluppo: servono a definire i *work package* e a sviluppare il sistema.
- I **diagrammi** rappresentano un'aggregazione *logica* dell'informazione: servono a capire il dominio.

### 2.3.11 Design pattern

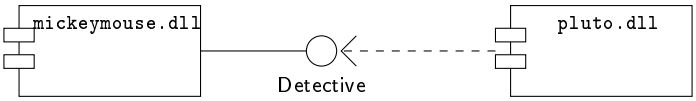
> *... in molti sistemi object-oriented si trovano pattern ricorrenti di classi e di oggetti che comunicano. Questi pattern risolvono specifici problemi di progettazione e rendono il design OO più flessibile, elegante e riusabile. Aiutano i progettisti a riusare design di successo basando i nuovi sull'esperienza passata. Un progettista che li conosce può applicarli subito, senza doverli riscoprire.* — Gamma et al.

Un design pattern è descritto da:

- **nome**: un'astrazione che comunica significato, applicabilità e intento;
- **descrizione del problema**: l'ambiente e le condizioni che devono esistere perché il pattern sia applicabile;
- **caratteristiche**: gli attributi del design che si possono adattare per applicare il pattern a problemi diversi;
- **conseguenze**: le ricadute delle scelte di design fatte usando il pattern.

### 2.3.12 Componenti e nodi

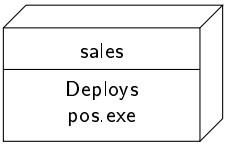
**Componenti** Un **componente** rappresenta l'impacchettamento *fisico* di elementi altrimenti logici (es. una `.dll`). Si disegna come un rettangolo con due rettangolini sul lato sinistro. Un componente può esporre **interfacce** (notazione a «lecca-lecca»), da cui altri componenti dipendono:




`mickeymouse.dll` *realizza* l'interfaccia Detective; `pluto.dll` *dipende* da essa (freccia tratteggiata). Le classi realizzate da un componente si collegano a esso con una dipendenza.

UML definisce cinque **stereotipi** per i componenti: *executable*, *library*, *table*, *file*, *document*. Si possono anche definire icone proprie.

**Nodi** Un **nodo** è un elemento fisico che esiste a run-time e rappresenta una risorsa computazionale, in genere con almeno un po' di memoria e spesso capacità di elaborazione. Si disegna come un parallelepipedo; può avere un nome semplice (`kiosk_7`) o esteso (`server::backup`), e può elencare i componenti che ospita (`Deploys pos.exe`).




#### Componenti vs. nodi

- Si somigliano: hanno nomi, partecipano a relazioni di dipendenza, generalizzazione e associazione, possono essere annidati, avere istanze, partecipare a interazioni.
- Però i **componenti** *partecipano* all'esecuzione del sistema, mentre i **nodi** *eseguono* i componenti.
- I componenti rappresentano l'impacchettamento fisico di elementi logici; i nodi rappresentano il *deployment* fisico dei componenti.

Un **diagramma di deployment** mostra i nodi (es. : `kiosk`, `s : server`, `c : console`), i componenti che ciascuno ospita e i collegamenti fisici tra nodi (es. «10-T Ethernet», «RS-232»).

### 2.3.13 UML per sistemi real-time

Per i sistemi real-time in UML vanno considerati due aspetti: le caratteristiche di UML già utilizzabili e le estensioni proposte (sfruttando la capacità di UML di auto-estendersi).

#### Costrutti real-time in UML

- **Eventi temporali e di cambiamento**: specificano quando un'azione va eseguita.
  - `when` indica la data/ora in cui l'evento avverrà: `when(12:00) / goForLunch()`;
  - `after` indica dopo quanto tempo l'evento avverrà: `after(5 minutes) / put the egg in the pot`.

  Si usano come eventi nelle transizioni degli statechart (es. da Resting: `after(1 hour) / check the house`, `when(12:00 noon) / go for lunch`).

- **Vincoli temporali** sui messaggi:
  - tre variabili predefinite: `startTime`, `stopTime`, `executionTime`;
  - il costrutto `every` indica un messaggio rinviato a intervalli regolari.




Purtroppo nessuno strumento o linguaggio di programmazione esistente fa rispettare questi vincoli.

**ROOM** (*Real-time Object Oriented Modeling*) è un'estensione per il real-time: introduce le «capsule» (oggetti attivi) che comunicano attraverso «port» tipizzate da un protocollo (es. `portA : ProtocolA::master`).

### 2.3.14 Object Constraint Language (OCL)

OCL è il linguaggio di espressioni di UML:

- **puro**: nessun effetto collaterale;
- di **modellazione**: non può essere eseguito;
- **formale**: ha una semantica ben definita.

Obiettivo: definire in modo pulito e non ambiguo le condizioni (es. le guardie) in UML.

Classe di esempio: Dog con `Real weight`, `Real age`, `eat(f: Real)`, `bark()`, associata a Person (ruoli `owner` e `pet`, molteplicità `*` sui cani); istanze Joe (persona) e Fido (cane).

#### Caratteristiche principali

- Gli attributi hanno la stessa sintassi di Java, ma `this` diventa `self`: `self.weight < 100`.
- Le espressioni sono **fortemente tipate** e la conformità dei tipi viene controllata: `self.weight < "abc"` è un errore.
- **Pre- e post-condizioni** sulle operazioni:

```
Dog::bark()
  pre:  neighborhood is happy
  post: neighborhood is angry
Dog::eat(f: Real)
  post: weight = weight@pre + f
```

  `@pre` indica il valore prima dell'esecuzione.

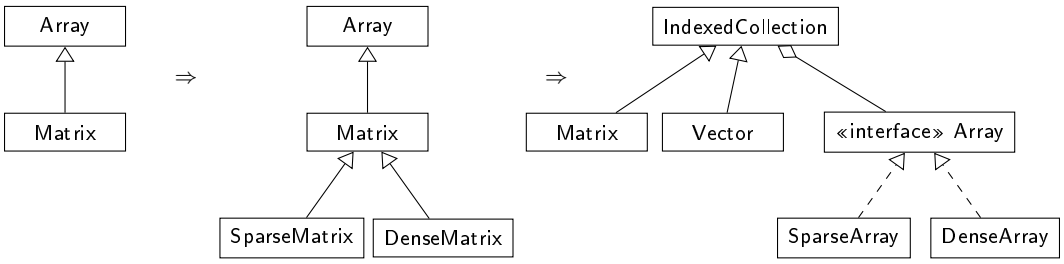
- Le associazioni con cardinalità 1 sono trattate come attributi (`Fido.owner = Joe`); quelle con cardinalità diversa da 1 come **insiemi** (`Joe->pet` è l'insieme dei cani di Joe). `.` e `->` si possono usare in modo polimorfo.
- Tipi: `OCLAny` (radice), `Real`, `Integer`, `String`, `Boolean`, enumerazioni, `OCLType`, e `Collection` con le sottoclassi `Set`, `Bag`, `Sequence`.
- Metodi delle collezioni: `size`, `includes`, `count`, `includesAll`, `isEmpty`, `notEmpty`, `sum`, `exists`, `forAll`, `iterate`, `select`, `reject`, `collect`.

### 2.3.15 Refactoring

I sistemi software evolvono e richiedono aggiornamento e «manutenzione» continui: serve renderli semplici (o almeno «non orribilmente difficili»). Da qui lo sviluppo incrementale con cicli brevi e **refactoring costante**.

Fare refactoring dei diagrammi UML significa rivedere il sistema ristrutturandone il design e quello dei suoi componenti, puntando alla semplicità e a una maggiore aderenza alla struttura del dominio, **mantenendone il comportamento**.

**Esempio**




Si parte da Matrix sottoclasse di Array; si aggiungono le specializzazioni; infine si separa ciò che si indicizza (IndexedCollection, con le sottoclassi Matrix e Vector) dalla strategia di memorizzazione (interfaccia Array, realizzata da SparseArray e DenseArray).

**Tecniche** Operazioni applicabili a classi, dati (statici) membro, funzioni (statiche) membro e dati temporanei:

| | Classe | Dati membro | Funzioni membro | Dati temp. |
|---|---|---|---|---|
| Aggiungere | ✓ | ✓ | ✓ | ✓ |
| Eliminare | ✓ | ✓ | ✓ | ✓ |
| Rinominare | ✓ | ✓ | ✓ | ✓ |
| Spostare su/giù | ✓ | ✓ | ✓ | — |
| Scomporre/condensare | ✓ | — | ✓ | — |
| Ri-qualificare | — | — | ✓ | ✓ |

(— = combinazione non applicabile.)

**Design pattern e refactoring** I design pattern aiutano il refactoring perché suggeriscono la struttura interna del codice, guidano la creazione di nuove strutture di oggetti e classi, e separano le strategie di implementazione dall'obiettivo. Guida:

- creazione coerente tra classi → **Abstract Factory**;
- creazione che dipende dall'ambiente → **Factory Method**;
- creazione a partire da un'entità prototipo → **Prototype**;
- numero variabile di funzionalità → **Decorator**;
- struttura ad albero → **Composite**;
- double dispatching → **Visitor**;
- implementazione flessibile di un algoritmo → **Strategy**;
- implementazione flessibile di una classe → **Bridge**.

**Criteri**

- fare refactoring quando il programma mostra carenze, non per «miglioramento concettuale»;
- scrivere il codice esattamente una volta (cfr. la relazione «include» nei casi d'uso);
- eliminare entità grandi (metodi, classi, package/namespace).

**Inibitori**: «il sistema funziona» e mancano tempo e soldi; dover lavorare con le astrazioni di design di qualcun altro.